

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS

VANESSA LUCION

**SISTEMA WEB PARA CONTROLAR O BALANCEAMENTO
NUTRICIONAL DO GADO LEITEIRO**

TRABALHO DE CONCLUSÃO DE CURSO

PATO BRANCO
2020

VANESSA LUCION

**SISTEMA WEB PARA CONTROLAR O BALANCEAMENTO
NUTRICIONAL DO GADO LEITEIRO**

Trabalho de Conclusão de Curso de graduação, apresentado à disciplina de Trabalho de Conclusão de Curso, do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas, da Universidade Tecnológica Federal do Paraná, Campus Pato Branco, como requisito parcial para obtenção do título de Tecnólogo.

Orientadora: Profa. Andréia Scariot Beulke

PATO BRANCO
2020

Pés, para que os quero, se tenho asas para voar!

Frida Kahlo

RESUMO

O maior desafio da produção de leite é fornecer ao animal o alimento adequado e de qualidade, para obter o máximo do potencial de produção de leite e ao mesmo tempo obter melhor relação entre o custo de produção e a lucratividade da propriedade. O alimento fornecido, mesmo sendo de qualidade, pode não condizer com a necessidade nutricional do animal, podendo sobrar, faltar ou o nutriente do alimento fornecido não ser o adequado para a necessidade do animal. Sistemas computacionais podem ser utilizados para armazenar dados históricos de produção de leite, auxiliar no gerenciamento do processo de produção e, entre outros, serem utilizados para definir automaticamente a quantidade e o tipo adequado ou ideal de alimento para cada animal, em suas diversas fases produtivas. Os sistemas *web* facilitam o acesso, o armazenamento e a consulta de dados, facilitando a análise e a tomada de decisão. O sistema proposto como resultado deste trabalho visa auxiliar no controle do balanceamento nutricional dos animais. Esse controle é realizado por meio dos dados coletados desde o nascimento, passando por todas as fases da vida, estipulando a alimentação adequada para cada fase, baseada no tipo de alimento disponível e nas necessidades nutricionais dos animais.

Palavras-chave: Balanceamento nutricional. Sistema web. Linguagem Java. Bovinocultura. Pecuária de leite.

ABSTRACT

The biggest challenge in milk production is to provide the animal adequate and quality feed to obtain the maximum potential for milk production, while the costs of producing this feed do not interfere with the profitability of the property. However, frequently, the feed provided does not fit with the animal nutritional necessity, it can be left, not be enough or the nutrient in the feed provided is not adequate for the animal's needs. Computational systems can be used to store milk production historical data, help the production process management and, between others, to be used to automatically define the quantity and the suitable or ideal types of feed for each animal, in its various productive phases. The web system makes access, storage and data search easier, facilitating analysis and decision making. The system proposed as a result of this work aims to assist in controlling the nutritional balance of the animals. Through the data collected since the birth, going through all stages of life, stipulating the adequate nutrition of each animal's life stage.

Keywords: Nutritional balance. Web system. Java language. Cattle farming. Dairy livestock.

LISTA DE FIGURAS

Figura 1 - <i>Model-View-Controller</i>	16
Figura 2 – Diagrama de casos de uso	26
Figura 3 – Diagrama Entidade-Relacionamento	29
Figura 4 – Tela de Login.....	31
Figura 5 – Cadastro de usuário	32
Figura 6 – Lista de propriedades	33
Figura 7 – Lista de técnicos ativos	33
Figura 8 – Cadastro de propriedade e usuário	33
Figura 9 – Lista de animais da propriedade.....	35
Figura 10 – Detalhes dos animais.....	35
Figura 11 – Cadastro de inseminação	36
Figura 12 – Cadastro de reprodução.....	37
Figura 13 – Escolha da situação do nascimento	37
Figura 14 – Cadastro de bezerro fêmea	37
Figura 15 – Cadastro de reprodução	38
Figura 16 – Lista de filhos.....	39
Figura 17 – Cadastro de pesos ou lactações do rebanho	39
Figura 18 – Cadastro para cálculo das exigências nutricionais	40
Figura 19 – Escolha dos alimentos	40
Figura 20 – Tela para incluir a quantidade de alimento.....	41
Figura 21 – Menu lateral	42
Figura 22 – Formulações agrupadas por data.....	42
Figura 23 – Relatório do balanceamento	43
Figura 24 – Menu lateral do administrador	43
Figura 25 – Listagem dos pesos esperados.....	44
Figura 26 – Lista de usuários do sistema.....	44
Figura 27 – Tela de permissões do usuário.....	45
Figura 28 – Estrutura do projeto.....	46

LISTA DE QUADROS

Quadro 1 - Lista de ferramentas e tecnologias	18
Quadro 2 – Manter usuários	21
Quadro 3 - Manter propriedade	21
Quadro 4 – Manter animais	21
Quadro 5 – Atualizar fase do animal	22
Quadro 6 - Manter peso dos animais	22
Quadro 7 – Calcular GMD	22
Quadro 8 - Manter composição dos alimentos	23
Quadro 9 – Manter forragens.....	23
Quadro 10 – Manter vacas em produção	23
Quadro 11 – Manter vacas em reprodução	23
Quadro 12 – Apta a reprodução	24
Quadro 13 – Lançar alimentos.....	24
Quadro 14 – Calcular exigência	24
Quadro 15 - Realizar balanceamento	24
Quadro 16 – Gerar prontuário	25
Quadro 17 – Gerar relatórios	25
Quadro 18 – Casos de uso	25
Quadro 19 - Operação de inclusão dos casos de uso de cadastro.....	27
Quadro 20 - Operação de exclusão dos casos de uso de cadastro	27
Quadro 21 - Operação de alteração dos casos de uso de cadastro.....	27
Quadro 22 - Operação de consulta dos casos de uso de cadastro.....	28
Quadro 23 - Caso de uso escolher alimentos.....	28

LISTAGENS DE CÓDIGO

Listagem 1 – Controller das raças	46
Listagem 2 – Classe de configuração Spring Security	47
Listagem 3 – Classe de usuário	49
Listagem 4 – Método lista permissões e acessos	50
Listagem 5 – Lista de técnicos ativos	51
Listagem 6 – Salva novo cadastro de usuário	52
Listagem 7 – Método para salvar uma propriedade	54
Listagem 8 – Método para vincular técnico a propriedade	54
Listagem 9 – Lista os detalhes do animal	55
Listagem 10 – Método para verificar fase do animal	56
Listagem 11 – Cálculo dos dias em lactação	57
Listagem 12 – Método para salvar uma nova inseminação	58
Listagem 13 – Método para salvar o nascimento de uma fêmea	60
Listagem 14 – Método para salvar o nascimento de um boi	60
Listagem 15 – Método para salvar prontuário	61
Listagem 16 – Calcular o GMD	62
Listagem 17 – Calcular o status do GMD	62
Listagem 18 – Método para verificar se está apta para reprodução	63
Listagem 19 – Método para verificar a fase do animal	63
Listagem 20 – Método para salvar lista de pesos	64
Listagem 21 – Exigência de matéria seca	65
Listagem 22 – Exigência de energia líquida	67
Listagem 23 – Exigência de nutrientes digestíveis totais	68
Listagem 24 – Exigência de energia metabolizável	68
Listagem 25 – Exigência de proteína degradável no rúmen	68
Listagem 26 – Exigência de proteína não degradável no rúmen	69
Listagem 27 – Exigência de proteína metabolizável	70
Listagem 28 – Exigência de proteína bruta	70
Listagem 29 – Exigência de cálcio	71
Listagem 30 – Exigência de fósforo	71
Listagem 31 – Cálculo do peso estimado para o próximo balanceamento	72
Listagem 32 – Cálculo dos dias em gestação	72
Listagem 33 – Cálculo dos dias em lactação	73
Listagem 34 – Cálculo do leite corrigido	73
Listagem 35 – Cálculo do peso estimado ao nascimento	74
Listagem 36 – Cálculo da produção de leite projetada	74
Listagem 37 – Lista alimentos para escolha	75
Listagem 38 – Método para salvar as formulações/alimento	75
Listagem 39 – Lista as formulações/alimentos	76
Listagem 40 – Cálculo dos nutrientes oferecidos	77
Listagem 41 – Requisição Ajax de formulação/alimento	78
Listagem 42 – Método para salvar as quantidades de alimentos	80
Listagem 43 – Cria lista de pesos do animal	80
Listagem 44 – Gerador do gráfico	81

LISTA DE SIGLAS

AJAX	Asynchronous JavaScript and XML
CNF	Carboidratos Não Fibrosos
CRUD	<i>Create, Retrieve, Update and Delete</i>
EE	Extrato etéreo
eFDN	Fibra em Detergente Neutro Efetiva
ELL	Energia Líquida
EM	Energia Metabolizável
DA	Fibra em Detergente Ácido
FDN	Fibra em Detergente Neutro
GMD	Ganho Médio Diário
IA	Inseminação Artificial
IDE	<i>Integrated Developmnet Environment</i>
HTML	<i>Hypertext Markup Language</i>
MS	Matéria Seca
NDT	Nutrientes Digestíveis Totais
PB	Proteína Bruta
PDR	Proteína Degradável no Rúmen
PM	Proteína Metabolizável
PNDR	Proteína Não Degradável No Rúmen
ppm	Parte por Milhão
MVC	<i>Model-View-Controller</i>
NRC	<i>National Research Council</i>
RIA	<i>Rich Internet Application</i>
Vit. A U.I./KG	Vitamina A (unidade internacional/kg)

SUMÁRIO

1 INTRODUÇÃO	10
1.1 CONSIDERAÇÕES INICIAIS	10
1.2 OBJETIVOS	11
1.2.1 Objetivo Geral	11
1.2.2 Objetivos Específicos	11
1.3 JUSTIFICATIVA	12
1.4 ESTRUTURA DO TRABALHO	13
2 REFERENCIAL TEÓRICO	14
2.1 BALANCEAMENTO NUTRICIONAL	14
2.2 DESENVOLVIMENTO DE APLICAÇÕES WEB	15
2.3 RICH INTERNET APPLICATION.....	16
3 MATERIAIS E MÉTODO.....	18
3.1 MATERIAIS	18
3.2 MÉTODO	19
4 RESULTADOS.....	20
4.1 ESCOPO DO SISTEMA	20
4.2 MODELAGEM DO SISTEMA	21
4.2.1 Requisitos funcionais e não-funcionais	21
4.2.2 CASOS DE USO	25
4.2.2.1 Casos de uso expandidos	26
4.2.3 Diagrama Entidade-Relacionamento	29
4.3 APRESENTAÇÃO DO SISTEMA	31
4.4. IMPLEMENTAÇÃO DO SISTEMA	45
CONCLUSÃO.....	82
REFERÊNCIAS	83

1 INTRODUÇÃO

Este capítulo apresenta as considerações iniciais, os objetivos e a justificativa da realização deste trabalho e a organização do texto com a apresentação dos capítulos.

1.1 CONSIDERAÇÕES INICIAIS

O agronegócio é o principal responsável pelo Produto Interno Bruto (PIB) do Brasil, correspondendo a 23%, envolvendo as atividades primárias, de transformação e distribuição do setor (PODESTÀ, 2017). Dentro desse setor existe uma dinâmica de cultivo da terra conhecida como agricultura familiar, que tem a principal forma de sustento proveniente da agropecuária em pequenas propriedades, sendo gerida apenas por integrantes da família (MDA, 2016).

Uma das atividades que mais estão presentes na agricultura familiar é a produção de leite bovino, sendo que 47% da produção é proveniente dessas pequenas propriedades (ZOCCAL, 2017).

O maior desafio da produção de leite é fornecer ao animal o alimento adequado e de qualidade para obter o máximo da capacidade de produção de leite, ao mesmo tempo que os custos da produção desse alimento não interfiram na lucratividade da atividade. Porém, muitas vezes, o alimento fornecido não condiz com a necessidade nutricional do animal, podendo sobrar, faltar ou o nutriente do alimento fornecido não ser o adequado para a necessidade do animal. Fórmulas matemáticas podem ser utilizadas para obter a melhor nutrição para o animal com o menor custo, ou seja, para obter a melhor relação custo x benefício para o produtor.

Diante desse desafio, o Instituto de Desenvolvimento Rural do Paraná - IAPAR-EMATER (IDR-Paraná) busca, por meio de uma unidade demonstrativa da propriedade de leite baseada em pasto, realizar experimentos com a alimentação dos animais, coletando dados de produção de leite, peso, quantidade e tipos de alimentos fornecidos, entre outras informações.

Os dados coletados mensalmente, que são em grande quantidade, são manipulados em planilhas de Excel, o que torna morosa a sua análise. Para emissão de relatórios e gráficos as informações precisam ser separadas planilha por planilha. A possibilidade de erro no lançamento dos dados prejudica a confiabilidade do resultado, pois não existem mecanismos

de validação. Além da quantidade de campos presentes em cada arquivo, tornando o ambiente de trabalho complexo.

Para a solução deste problema, propõe-se a construção de um sistema *web* que armazene os dados coletados para controlar o balanceamento nutricional dos animais. O sistema será alimentado com dados dos animais desde seu nascimento, passando por todas as fases da vida, estipulando a alimentação adequada para cada fase, baseada no tipo de alimento disponível. O sistema será implantado, inicialmente, dentro da instituição de pesquisa, para teste e adequações e, posteriormente, sendo de interesse da entidade, será implantado nas propriedades da agricultura familiar acompanhadas pela equipe de pesquisadores.

1.2 OBJETIVOS

Os objetivos se dividem em geral e específicos. O objetivo geral está relacionado ao sentido mais amplo da realização deste trabalho e os objetivos específicos complementam o geral por meio de funcionalidades do sistema.

1.2.1 Objetivo Geral

Desenvolver um sistema web para controlar o balanceamento nutricional do gado leiteiro.

1.2.2 Objetivos Específicos

- Realizar a avaliação do ganho de peso.
- Verificar se as novilhas estão aptas para reprodução.
- Realizar o cálculo dos nutrientes oferecidos em relação a quantidade de alimento volumoso, concentrado e mineral e os cálculos de exigência nutricional do animal.
- Armazenar os dados e comparar os níveis nutricionais exigidos pelo animal e fornecidos pelo alimento disponibilizado.
- Emitir relatório do balanceamento nutricional das vacas em lactação e gráficos da evolução do peso e da produção de leite.

1.3 JUSTIFICATIVA

Diante da necessidade de melhorar a coleta, o armazenamento e a análise de dados dos mais variados tipos, sejam de uso comercial, empresarial ou institucional, é importante ter o auxílio de Sistemas de Informação (SI). Eles permitem que as informações coletadas sejam armazenadas em banco de dados, facilitando, assim, a busca e a consulta desses dados para acompanhamento, análise e tomada de decisão. Assim, esses sistemas permitem o gerenciamento das atividades rotineiras de uma instituição, por exemplo.

Com relação aos dados gerados pela agricultura familiar, especialmente de gado leiteiro, é necessário que sejam confiáveis para prever e medir a quantidade de alimento ingerido diariamente permitindo que seja estimado o consumo de distintas categorias em diferentes períodos do ano e do ciclo produtivo. Os dados são utilizados para controlar se as novilhas estão aptas a reprodução, a quantidade de volumoso, concentrado e o início do fornecimento de silagem para bezerras e novilhas, calcular a composição da ração para bezerras e novilhas, calcular e comparar os níveis nutricionais exigidos pelo animal e fornecidos pelo alimento disponibilizado, além de calcular indicadores financeiros da produção de leite, indicadores zootécnicos e estimativa de produção de leite e eficiência leiteira de cada animal, por exemplo, dentre outras finalidades.

Todas as tarefas citadas exigem dados precisos e confiáveis para que o resultado seja satisfatório e que auxilie na tomada de decisão mais adequada. As planilhas eletrônicas são capazes de armazenar esses dados e fazer os cálculos, mas podem ser suscetíveis a erros se não operadas de forma adequada, além de ser trabalhoso manipular os dados para obter as informações desejadas.

Um SI permite o registro de todas as informações necessárias para realizar os cálculos e para emitir relatórios do balanceamento de vacas em lactação e secas, entre outros. Esses dados serão armazenados em um banco de dados que possui um conjunto de tabelas relacionadas e serão disponibilizados para consultas e análises pelo produtor e assistência técnica.

Diante destas justificativas, a realização desse trabalho vem a contribuir para outros trabalhos de desenvolvimento *web* para agricultura. Visto que o agronegócio é uma área promissora e com amplas possibilidades de inovação tecnológica.

O que motiva desenvolver o sistema proposto é a possibilidade de levar para uma Instituição pública de pesquisa, que desenvolve a agricultura em prol dos interesses da sociedade, uma ferramenta que visa fornecer agilidade nos processos, confiabilidade no

registro e na análise dos dados, além de auxiliar no desenvolvimento dos programas de pesquisa.

1.4 ESTRUTURA DO TRABALHO

Este texto está organizado em capítulos e este é o primeiro e apresenta a introdução, as considerações iniciais, os objetivos e a justificativa de realização deste trabalho. O Capítulo 2 apresenta o referencial teórico. No Capítulo 3 são apresentados os materiais e o método utilizados para o desenvolvimento do trabalho. No Capítulo 4 está o resultado da realização do trabalho que apresenta a modelagem e a implementação do sistema. Por fim, está a conclusão do trabalho e as referências utilizadas para o desenvolvimento.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta a fundamentação teórica deste trabalho, cujo conteúdo explana sobre balanceamento nutricional e desenvolvimento de aplicações web.

2.1 BALANCEAMENTO NUTRICIONAL

Dentro do sistema produtivo da atividade leiteira, a nutrição do rebanho é fator primordial para se obter uma maior produtividade e qualidade no produto, com o custo menor (TOMICHI et al., 2015). Em relação ao custo com alimentação, Salman, Osmari e Santos (2011) relatam que representa “40% a 60% ou até mais do custo total de produção”. Com isso, “o planejamento adequado do manejo nutricional do rebanho evita gastos desnecessários e favorece os lucros com a atividade” (SALMAN; OSMARI; SANTOS, 2011, p. 7).

Para que o planejamento ocorra, é necessário realizar o balanceamento da dieta de cada animal e, assim, fornecer o alimento necessário, na quantidade e mistura adequadas para se obter um determinado desempenho (LAZIA, 2012).

Para realizar o balanceamento, são necessárias as seguintes etapas (SALMAN; OSMARI; SANTOS, 2011):

- 1) Caracterizar os animais para os quais se deseja balancear a ração.
- 2) Verificar as exigências dos nutrientes dos animais de acordo com o item anterior.
- 3) Levantar e quantificar os alimentos disponíveis.
- 4) Relacionar a composição química e o valor energético dos alimentos a serem utilizados, considerando-se os nutrientes de interesse.
- 5) Proceder ao balanceamento da ração para a proteína bruta e energia.
- 6) Depois de concluído o cálculo da ração, verificar se todas as exigências foram atendidas.

A exigência nutricional é definida por Salman, Osmari e Santos (2011, p. 15) “como sendo a quantidade de cada nutriente necessária para manutenção, crescimento, reprodução e produção de uma determinada espécie ou categoria animal”. Também, que cada animal apresenta uma exigência nutricional diária, que é influenciada pelo nível de produção, peso corporal e estágio fisiológico que se encontra.

Para calcular a exigência nutricional do gado leiteiro existem normas e padrões definidos por cada país, conforme suas características (TOMICHI et al., 2015). No Brasil, por

não existir uma definição, a tabela *National Research Council* (NRC) dos Estados Unidos, atualizada em 2001, é a mais utilizada (COSTA et al., 2012).

Para atender à exigência do animal, é necessário escolher os alimentos conforme seu valor nutritivo. Para isso, cada alimento deve conter sua composição química, que definirá a sua quantidade no balanceamento (TOMICICH et al., 2015).

Diante da quantidade de dados que são processados para se chegar a um resultado, mostra a importância de um sistema de informação, que armazene e analise os registros, trazendo praticidade e rapidez ao trabalho.

2.2 DESENVOLVIMENTO DE APLICAÇÕES WEB

Para Bianchini (2008) uma aplicação web fornece, basicamente, as mesmas funcionalidades que uma aplicação de software tradicional, pois envolve características próprias relacionadas à web, como interação entre cliente e servidor e características comuns de outros tipos de aplicações como a implementação de regras de negócio e banco de dados.

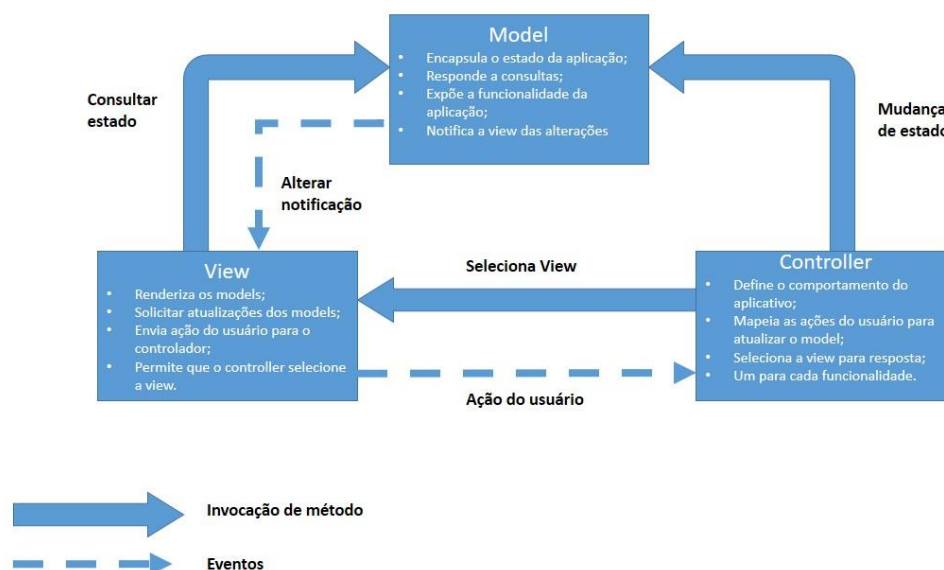
Ainda, para que uma aplicação web seja desenvolvida de forma adequada, é importante conhecer sua arquitetura que possui três componentes principais: servidor, cliente e conexão de rede que compõe uma arquitetura em três camadas. Essas camadas são responsáveis por separar as funções de interface com o usuário, acesso ao banco de dados e lógica de negócio (BIANCHINI, 2008).

Um exemplo de arquitetura em três camadas utilizada no desenvolvimento web é o *Model-View-Controller* (MVC). Esse padrão permite definir o papel de cada parte da aplicação, tornando-a independente (OLIVEIRA JUNIOR; FORTES, 2007).

Na camada *model* ficam “os objetos que implementam a funcionalidade principal do sistema de informação”, (DURELLI; VIANA; PENTEADO, 2008, p. 4), bem como, criam as entidades no banco, para armazenar os dados (OLIVEIRA JUNIOR; FORTES, 2007). A camada *view* representa a interface gráfica, usada pelo usuário para interagir com o sistema, também é onde os dados serão apresentados (DURELLI; VIANA; PENTEADO, 2008). Já a camada *controller* “define a maneira como a *view* deve agir, a partir das informações fornecidas pelo usuário, além de atualizar as informações e o estado dos objetos da camada *model*” (DURELLI; VIANA; PENTEADO, 2008, p. 4).

Na Figura 1, pode-se visualizar como ocorre a interação entre o *Model-View-Controller*:

Figura 1 - Model-View-Controller



Fonte: Adaptado de Oliveira Junior e Fortes (2017, p.5).

A camada de *controller* interpreta as entradas fornecidas pela *view* e faz o mapeamento das ações do usuário que são enviadas para a *model* e/ou para a *view*. A *model* faz o gerenciamento de um ou mais elementos de dados, responde as perguntas sobre seu estado e instruções para mudá-lo. A *model* é a principal estrutura da arquitetura, pois é ela quem modela o problema a ser resolvido. Por fim, a *view* é responsável pela apresentação do conteúdo.

2.3 RICH INTERNET APPLICATION

Com a popularização da internet, a facilidade de acesso à informação, a praticidade de utilizar um sistema de qualquer lugar, tem feito com que muitos sistemas *desktop* migrem para web ou sejam desenvolvidos especificamente para web. Para que a interação do usuário com o sistema permaneça a mesma, ou bem semelhante, foram criadas as Aplicações Ricas para Internet (*Rich Internet Application* – RIA).

São “numerosas aplicações que dependem, pesadamente, da interação rica com o usuário para completar tarefas do dia-a-dia, elas são desenvolvidas como aplicações cliente/servidor” (SMEETS; BANKRAS, 2009).

As Aplicações Ricas para Internet utilizam um modelo onde o conteúdo da camada de apresentação, parte gráfica da aplicação, é armazenado e executado no lado do cliente, reduzindo significativamente a comunicação com o servidor. A aplicação

cliente é capaz de realizar operações e cálculos, enviar e receber dados em segundo plano, de forma assíncrona. Desta forma, ao interagir com a aplicação, não se faz necessário que a página seja redesenhada por completo quando solicitado novos dados ao servidor, assim, o usuário obtém uma resposta quase imediata da aplicação e durante este processo pode continuar interagindo normalmente com a mesma (DUHL, 2003; LÓPEZ, 2005; LUNA, 2009 apud MATUZAKI, 2013, p.19).

Para “manipular o conteúdo, a estrutura e o estilo de um documento *web* em tempo de carregamento ou interação do usuário com a página” é usada a combinação de HTML, com JavaScript e CSS. Junto com essas tecnologias, o *Asynchronous JavaScript and XML (AJAX)* agrega a “interface `xmlhttprequest`, do javascript, para enviar requisições http assíncronas”. Possibilitando que o usuário continue interagindo com o sistema, mesmo durante o processamento da requisição (LUNA, 2009, p.24).

3 MATERIAIS E MÉTODO

Este capítulo apresenta os materiais e o método utilizados para a realização deste trabalho. Os materiais estão relacionados às tecnologias e ferramentas utilizadas e o método apresenta a sequência das principais atividades realizadas.

3.1 MATERIAIS

As tecnologias que serão utilizadas no desenvolvimento do trabalho estão relacionadas no Quadro 1.

Quadro 1 - Lista de ferramentas e tecnologias

Ferramenta/ Tecnologia	Versão	Finalidade	Disponível em
Apache Maven	3.3.9	Gerenciamento de dependências e <i>build</i> .	https://maven.apache.org
CSS	3	Estilização de conteúdos <i>Hypertext Markup Language</i> (HTML).	https://www.w3schools.com/css/
Hibernate	5.0.12	Mapeamento objeto-relacional para persistência e consulta de informações.	http://hibernate.org/orm/
HTML	5	Linguagem de marcação de conteúdos para desenvolvimento da interface da aplicação.	https://www.w3schools.com/html/
JasperReports	6.13.0	Ferramenta para geração de relatórios Java	https://community.jaspersoft.com/project/jasperreports-library
JavaScript		Linguagem de programação executada no lado cliente da aplicação.	https://www.w3schools.com/js/
Linguagem Java	1.8	Linguagem de programação orientada a objetos para a construção do projeto lado servidor.	http://www.oracle.com/technetwork/java/javase/downloads/jdk8-downloads-2133151.html
PgAdmin	1.22.2	Gerenciador de banco de dados.	https://www.pgadmin.org
PostgreSQL	9.6.2	Banco de dados relacional para armazenamento de informações.	https://www.postgresql.org
Spring Framework	5.0.0	Framework Java para gerenciar processos básicos permitindo foco na codificação da aplicação.	https://spring.io
Spring Tool Suite 4	4.1.0.RELEASE	<i>Integrated Development Environment</i> (IDE) de desenvolvimento	https://spring.io/tools
Thymeleaf	3.0.11	Mecanismo de modelo Java / HTML5	https://www.thymeleaf.org/
Visual	14.1	Modelagem do sistema.	https://www.visual-

Paradigm			paradigm.com/download/community.jsp
----------	--	--	--

Fonte: Autoria própria.

3.2 MÉTODO

O método consiste nas atividades de levantamento e modelagem de requisitos. O método utilizado neste trabalho foi o processo sequencial linear de Pressman (2008) que define as fases de levantamento de requisitos, análise e projeto e desenvolvimento do sistema. Neste trabalho são apresentadas as duas primeiras etapas.

Para o levantamento dos requisitos foram realizadas entrevista e análise da documentação da Instituição de pesquisa. A entrevista foi realizada com pesquisadores que atuam na formulação da dieta dos animais em três diferentes Polos de pesquisa. Também foram buscadas informações com o técnico responsável pelos animais do Polo de Pato Branco, sobre procedimentos rotineiros necessários no sistema.

A equipe utiliza diversas planilhas de Excel para registrar e manipular os dados dos projetos de pesquisa e propriedades que acompanham. Sendo esses arquivos usados para análise documental, podendo levantar os tipos de dados necessários para o desenvolvimento do sistema e como eles interagem para gerar informação.

Na sequência, foi realizada a análise e o projeto do sistema, definindo os requisitos como funcionais e não funcionais. Baseado nisso foi elaborado o diagrama de casos de uso e sua expansão, trazendo subsídios para realizar a modelagem do banco de dados.

Após isso, foi iniciado o processo de desenvolvimento das interfaces e dos códigos-fontes do sistema. Os testes foram realizados informalmente, conforme cada requisito era implementado.

4 RESULTADOS

Este capítulo apresenta o resultado deste trabalho que é o desenvolvimento de um sistema *web* para controlar e balanceamento nutricional do gado leiteiro. Inicialmente é apresentado o escopo do sistema que relata as principais funcionalidades e atores envolvidos, seguido da modelagem que estabelece a definição dos requisitos funcionais e não-funcionais, diagrama dos casos de uso e de entidade e relacionamento do banco de dados. A apresentação do sistema consiste em exibir as principais telas do sistema, seguido da exibição dos códigos-fontes desenvolvidos para o projeto.

4.1 ESCOPO DO SISTEMA

O sistema permite realizar o balanceamento nutricional, baseado nos alimentos que irão compor a dieta das vacas de uma propriedade. O sistema possibilita, ainda, controlar e analisar o desenvolvimento do animal, por meio dos registros de peso e da produção de leite.

O sistema possui três atores: o proprietário que coleta e lança os dados no sistema; o técnico que faz a análise desses dados, gera os balanceamentos e acompanha a evolução do rebanho da propriedade; o administrador que gerencia os cadastros gerais do sistema, as permissões dos demais usuários e os acessos às propriedades. Os dados para análise são fornecidos pelos cadastros e pelo processamento dos cálculos necessários. Os cadastros se referem à propriedade, com dados da área, da forragem que está disponível para o rebanho, da composição química dos alimentos; dos animais que nascem na propriedade (peso, data de nascimento, raça, número do brinco, nome); e dos animais já em produção, como a quantidade de leite, teor de gordura e proteína do leite e preço.

As principais funções do sistema são: avaliação do ganho de peso diário dos animais, se as novilhas estão aptas para reprodução; cálculo dos níveis nutricionais exigidos pelo animal (vaca) e a quantidade de nutrientes oferecidos através dos alimentos escolhidos (volumoso, concentrado e mineral); controle de datas de parto, pré-parto, secagem, inseminação e aborto; além de manter os registros no prontuário.

Os relatórios necessários para análise dos dados são o balanceamento nutricional das vacas e gráficos da curva de crescimento das bezerras e novilhas por meio do peso e da produção de leite das vacas em lactação.

4.2 MODELAGEM DO SISTEMA

Esta seção apresenta os requisitos funcionais e não funcionais e os diagramas usados para detalhar os processos e a estrutura do software desenvolvido. Cada requisito funcional é apresentado seguido dos requisitos não funcionais relacionados.

Os Quadros 2 a 18 apresentam os requisitos funcionais e não funcionais, respectivamente.

4.2.1 Requisitos funcionais e não-funcionais

Quadro 2 – Manter usuários

F1 Manter usuários	
Descrição: Inserir o cadastro de usuários, que terá campos de nome, telefone, e-mail, endereço, cidade, estado, entre outros.	
Requisitos Não-Funcionais	
Nome	Restrição
NF 1.1 Acesso ao sistema	O acesso ao sistema será realizado por meio de <i>login</i> e senha.
NF 1.2 Acesso aos dados	O acesso aos dados deve ser protegido por níveis de usuários.

Fonte: Autoria própria.

Quadro 3 - Manter propriedade

F2 Manter propriedade
Descrição: Inserir cadastro de propriedade, que terá os campos área total da propriedade, área de pasto, área de lavoura verão, área lavoura inverno, número de pessoas envolvidas na bovinocultura, localização.

Fonte: Autoria própria.

Quadro 4 – Manter animais

F3 Manter animais	
Descrição: Inserir e editar os animais da propriedade, seja por nascimento ou compra. Os dados cadastrados serão: peso, data de nascimento, raça, número do brinco, nome, pai, mãe, fase do animal, entre outros.	
Requisitos Não-Funcionais	
Nome	Restrição
NF 1.1 Animais comprados	Quando um animal for comprado, devem ser cadastrados os dados de fase do animal, brinco, aptidão para reprodução, data de inseminação, entre outros.
NF 1.2 Bois nascidos na propriedade	O sistema deve fazer inserção na tabela “bois_propriedade” os animais nascidos machos.
NF 1.3 Calcular a idade do	O sistema deve calcular a idade em dias.

animal	
--------	--

Fonte: Autoria própria.

Quadro 5 – Atualizar fase do animal

F4 Fase do animal	
Descrição: O sistema deve atualizar automaticamente em que fase da vida o animal se encontra.	
Requisitos Não-Funcionais	
Nome	Restrição
NF 1.1 Bezerras	A fase será bezerra quando for incluída data de nascimento do animal, e a idade ficar entre de 0 a 6 meses.
NF 1.2 Novilhas	A fase será novilha quando a idade for maior que 6 meses e ainda não tiver tido nenhum parto.
NF 1.3 Vaca em lactação	A fase será lactação após o registro do parto até ser lançada a data de secagem programada.
NF 1.4 Vaca seca	A fase será seca após a data de secagem até a data prevista de pré-parto.
NF 1.5 Vaca pré-parto	A fase será pré-parto a partir da data prevista do pré-parto até o registro do parto efetivo.

Fonte: Autoria própria.

Quadro 6 - Manter peso dos animais

F5 Manter peso dos animais	
Descrição: Inserir, editar e excluir o cadastro pesos dos animais, que terá os campos de escore condição corporal (ECC), data da pesagem, peso atual, idade, peso mínimo esperado, peso máximo esperado, ganho médio diário (GMD), <i>status</i> do GMD.	
Requisitos Não-Funcionais	
Nome	Restrição
NF 1.1 Escore condição corporal	Esse campo é de preenchimento obrigatório, ficando sob responsabilidade do técnico a inserção do valor adequado.

Fonte: Autoria própria.

Quadro 7 – Calcular GMD

F6 Calcular o GMD (ganho médio diário)	
Descrição: Sistema calculará o ganho de peso médio por dia do animal, conforme os pesos são inseridos.	
Requisitos Não-Funcionais	
Nome	Restrição
NF 1.1 Cálculo GMD	Para o cálculo utilizar: (peso atual - peso anterior) / (data atual - data pesagem anterior).
NF 1.2 Comparar	O resultado do GMD deve ser comparado ao GMD mínimo e GMD máximo permitidos, conforme tabela “pesos_esperados” do banco de dados.
NF 1.3 Resultado comparação	Após a comparação o sistema deve informar o <i>status</i> do GMD, se está normal, baixo ou alto.

Fonte: Autoria própria.

Quadro 8 - Manter composição dos alimentos

F7 Manter composição química dos alimentos
Descrição: Inserir, editar e excluir o cadastro de alimentos para o balanceamento nutricional. O cadastro conterá os seguintes campos: categoria (volumoso, concentrado, mineral), ingrediente, MS (%), PB (kg), PM (kg), PDR (kg), PNDR (kg), NDT (kg), EM (Mcal/kg), ELL (Mcal/kg), EE (kg), FDA (kg), FDN (kg), eFDN (kg), CNF (kg), Cinzas (kg), Ca (kg), P (kg), %Mg, %Na, %Cl, %K, %S, ppm Cu, ppm Zn, ppm Fe, ppm Mn, ppm I, ppm Co, ppm Se, Vit. A U.I./kg, Vit. D U.I./kg, Vit. E U.I./kg.

Fonte: Autoria própria.

Quadro 9 – Manter forragens

F8 Manter as variedades de forragens disponíveis	
Descrição: Inserir, editar e excluir o cadastro de forragens que estão disponíveis na propriedade para alimentação dos animais. O cadastro terá os campos: data, nome da forrageira (ex. aveia, grama tifton, coast croos), altura da forragem no momento da entrada em cm, resíduo na saída em cm, kg/m² em material natural (MN), área do piquete, eficiência em % (do pastejo), número de vacas, kg de MN/Vaca	
Requisitos Não-Funcionais	
Nome	Restrição
NF 1.1 Cálculo kg de MN/Vaca	O sistema deve calcular o kg de MN/Vaca utilizando a seguinte fórmula: ((tamanho do piquete * kg/m² em material natural (MN)) / (eficiência em %)) / número de vacas.

Fonte: Autoria própria.

Quadro 10 – Manter vacas em produção

F9 Manter vacas em produção	
Descrição: Inserir, editar e excluir o cadastro de produção de leite, com os campos data da medição, produção de leite (kg/dia), teor de gordura (%), teor de proteína (%), preço (R\$/L).	
Requisitos Não-Funcionais	
Nome	Restrição
NF 1.1 Calcular número de lactações anteriores	O sistema deve calcular o número de lactações a partir da quantidade de partos ocorridos.
NF 1.2 Calcular dias em lactação	O sistema deve calcular os dias que o animal está em lactação utilizando a data atual – data do parto.
NF 1.3 Calcular data secagem	O sistema deve fazer o cálculo da data de secagem do animal utilizando data prevista do parto - 60 dias.

Fonte: Autoria própria.

Quadro 11 – Manter vacas em reprodução

F10 Manter vacas em reprodução	
Descrição: Inserir, editar e excluir o cadastro de reprodução dos animais, com os campos data inseminação artificial (IA), touro, confirmação de prenhez (sim ou não), data de confirmação da prenhez, data do aborto, data da secagem, pré-parto e parto (quando efetivamente ocorreu).	
Requisitos Não-Funcionais	
Nome	Restrição

NF 1.1 Calcular data prevista para o parto	O sistema deve fazer o cálculo utilizando a data confirmação + 270 dias.
NF 1.2 Calcular data pré-parto	O sistema deve fazer o cálculo utilizando data prevista do parto – 20 dias.
NF 1.3 Calcular quantidade de inseminações	O sistema deve contar quantas inseminações já foram realizadas no animal.
NF 1.4 Lançamento de novas inseminações artificiais	Só poderá ser lançada nova IA, caso a data de aborto ou parto da última IA confirmada esteja lançada.

Fonte: Autoria própria.

Quadro 12 – Apta a reprodução

F11 Verificar se está apta a reprodução	
Descrição: O sistema deve informar ao usuário se o animal apresenta condições de peso e idade para reprodução.	
Requisitos Não-Funcionais	
Nome	Restrição
NF 1.1 Condições de peso e idade	O peso para animais de pequeno porte é de 250 kg, médio 300 kg e grande 350 kg. A idade para animais de pequeno porte é de 12 meses, médio e grande porte 13 meses. Emitir alerta ao usuário quando o animal estiver apto a reprodução.

Fonte: Autoria própria.

Quadro 13 – Lançar alimentos

F12 Lançar os alimentos (volumoso, concentrado, minerais) do balanceamento
Descrição: Usuário incluirá os alimentos que serão utilizados pelo sistema para fazer os cálculos para o balanceamento nutricional das vacas.

Fonte: Autoria própria.

Quadro 14 – Calcular exigência

F13 Calcular a exigência nutricional das vacas	
Descrição: o sistema deve calcular a exigência de matéria seca NRC (2001), de energia, de proteína, de proteína degradável no rúmen (PDR), de proteína não degradável no rúmen (PNDR), de cálcio, fósforo entre outros.	
Requisitos Não-Funcionais	
Nome	Restrição
NF 1.1 Filtrar animais	O cálculo da exigência nutricional será feito apenas para os animais em que a fase esteja como vacas em lactação, seca ou pré-parto.

Fonte: Autoria própria.

Quadro 15 - Realizar balanceamento

F14 Calcular a quantidade de nutrientes oferecidos no balanceamento
Descrição: O sistema deve calcular a quantidade de nutrientes oferecidos para balancear com a exigência nutricional de cada vaca.

Requisitos Não-Funcionais	
Nome	Restrição
NF 1.1 Filtro	O cálculo do oferecido será feito apenas para animais que a exigência nutricional foi calculada.

Fonte: Autoria própria.

Quadro 16 – Gerar prontuário

F15 Gerar prontuário
Descrição: O sistema deve permitir o registro de aplicação de medicamentos, tratamentos, doenças dos animais, cirurgias.

Fonte: Autoria própria.

Quadro 17 – Gerar relatórios

F16 Gerar relatórios	
Descrição: Os relatórios necessários para análise dos dados são: balanceamento nutricional das vacas. Os gráficos necessários são o de Peso e Lactação.	
Requisitos Não-Funcionais	
Nome	Restrição
NF 1.1 Data do relatório	O relatório será gerado na data que foi realizado o balanceamento.
NF 1.2 Período Gráfico	Os gráficos serão criados com os dados dos últimos 12 meses.

Fonte: Autoria própria.

4.2.2 CASOS DE USO

O Quadro 18 contém os casos de uso, seus atores e os requisitos funcionais que fazem parte de cada caso.

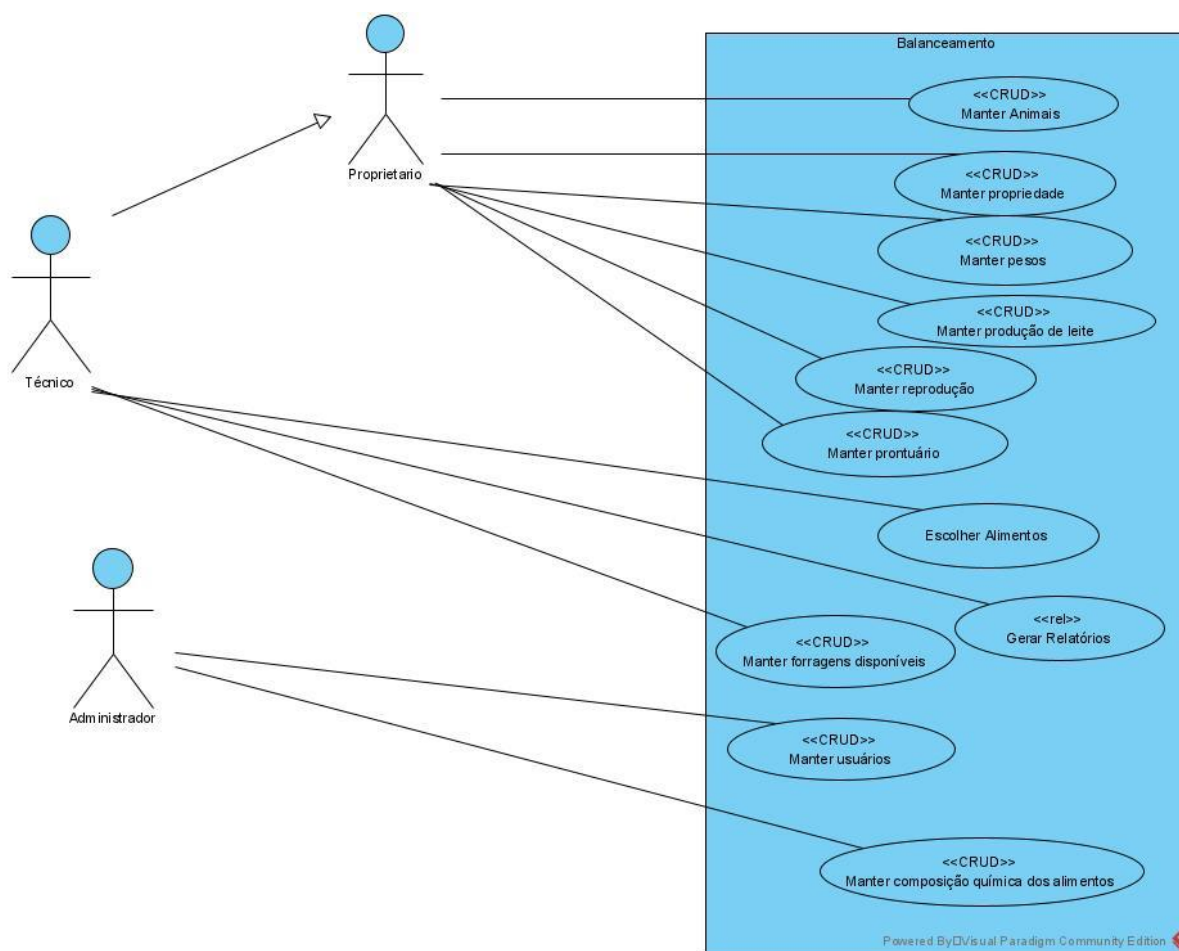
Quadro 18 – Casos de uso

ID	Nome do caso de uso	Atores	Referência Cruzada
UC1	Manter animais (<i>Create, Retrieve, Update and Delete</i> - CRUD)	Proprietário, Técnico	F3, F4
UC2	Escolher alimentos	Técnico	F12, F13, F14
UC3	Gerar relatórios (rel)	Técnico, Proprietário	F16
UC4	Manter produção de leite (CRUD)	Técnico, Proprietário	F9
UC5	Manter reprodução (CRUD)	Técnico, Proprietário	F10, F11
UC6	Manter prontuário (CRUD)	Técnico, Proprietário	F15
UC7	Manter pesos (CRUD)	Técnico, Proprietário	F5, F6
UC8	Manter composição química dos alimentos (CRUD)	Administrador	F7
UC9	Manter usuários (CRUD)	Administrador	F1
UC10	Manter propriedade (CRUD)	Técnico, Proprietário	F2
UC11	Manter as variedades de forragens disponíveis (CRUD)	Técnico	F8

Fonte: Autoria própria.

A Figura 2 apresenta o diagrama dos casos de uso do sistema que contém três atores denominados de proprietário, técnico e administrador. O proprietário é responsável por, além de manter os dados dos animais, manter os dados relacionados a peso, prontuário, reprodução e propriedade. O técnico é responsável por gerar relatórios, manter usuários e as forragens disponíveis, manter a composição química dos alimentos e escolher os alimentos. Por fim, o administrador atribui permissões aos usuários e o vincula a novas propriedades.

Figura 2 – Diagrama de casos de uso



Fonte: Autoria própria.

4.2.2.1 Casos de uso expandidos

Nos Quadros 19, 20, 21 e 22 são descritas as operações do tipo “manter” que se referem à inclusão, exclusão, alteração e consulta dos dados inseridos no banco de dados.

O Quadro 19 especifica a operação de inclusão de um novo registro.

Quadro 19 - Operação de inclusão dos casos de uso de cadastro

Caso de Uso: Incluir novo registro.
Atores: Proprietário, técnico e administrador de acordo com as funcionalidades do sistema.
Precondições: Estar autenticado no sistema.
Pós-condições: Novo registro incluído com sucesso.
Sequência típica de eventos (fluxo principal): Esse caso de uso inicia quando: 1. [IN] Ator seleciona tela para cadastro de novo registro. 2. [IN] Ator lança os dados do novo registro 3. [IN] Ator clica em salvar o novo registro 4. [OUT] Sistema grava os dados no banco e apresenta mensagem de confirmação.
Tratamento de Exceções e Variantes: Exceção: Campos obrigatórios não cadastrados 1 [OUT] Sistema verifica campos obrigatórios com valor <i>null</i>. 2 [OUT] Sistema emite mensagem de erro ao usuário, informando quais campos estão vazios. 3 [IN] Ator lança dados que faltam. 4 [IN] Ator clica em salvar o novo registro.

Fonte: Autoria própria.

O Quadro 20 especifica a operação de exclusão de um registro.

Quadro 20 - Operação de exclusão dos casos de uso de cadastro

Caso de Uso: Excluir registro (operação de exclusão)
Atores: Proprietário, técnico e administrador de acordo com as funcionalidades do sistema.
Precondições: Estar autenticado no sistema.
Pós-condições: Registro excluído com sucesso
Sequência típica de eventos (fluxo principal): 1. [IN] Ator acessa tela de consulta. 2. [IN] Ator pesquisa por registro que deseja excluir e seleciona. 3. [IN] Ator clica em excluir registro. 4. [OUT] Sistema pede confirmação para exclusão do registro. 5. [IN] Ator confirmar exclusão. 6. [OUT] Sistema excluir registro do banco de dados.
Tratamento de Exceções e Variantes: Exceção: Exclusão de registro com vínculos no sistema 1 [IN] Ator clica em excluir um registro que possui vínculos no sistema. 2 [OUT] O sistema verifica que o registro tem vínculos, não o exclui e exibe mensagem de alerta ao usuário

Fonte: Autoria própria.

O Quadro 21 especifica a operação de alteração de um registro.

Quadro 21 - Operação de alteração dos casos de uso de cadastro

Caso de Uso: Alterar registro
Atores: Proprietário, técnico e administrador de acordo com as funcionalidades do sistema.

Precondições: Estar autenticado no sistema.
Pós-condições: Registro alterado com sucesso
Sequência típica de eventos (fluxo principal): 1. [IN] Ator acessa tela de consulta. 2. [IN] Ator pesquisa por registro que deseja editar e seleciona. 3. [IN] Ator clica em editar registro. 4. [OUT] Sistema abre tela para edição dos dados salvos no banco. 5. [IN] Ator altera os dados e clica em salvar. 6. [OUT] Sistema salvar registro do banco de dados.

Fonte: Autoria própria.

O Quadro 22 especifica a operação de consulta de dados.

Quadro 22 - Operação de consulta dos casos de uso de cadastro

Caso de Uso: Consultar registros.
Atores: Proprietário, técnico e administrador de acordo com as funcionalidades do sistema.
Precondições: Estar autenticado no sistema.
Pós-condições: Relatório gerado com sucesso
Sequência típica de eventos (Fluxo Principal): 1. [IN] Ator acessa tela de gerar relatório. 2. [IN] Ator seleciona o relatório de deseja gerar. 3. [OUT] Sistema gera arquivo com os dados solicitado.

Fonte: Autoria própria.

O Quadro 23 especifica o caso de uso denominado escolher alimentos.

Quadro 23 - Caso de uso escolher alimentos

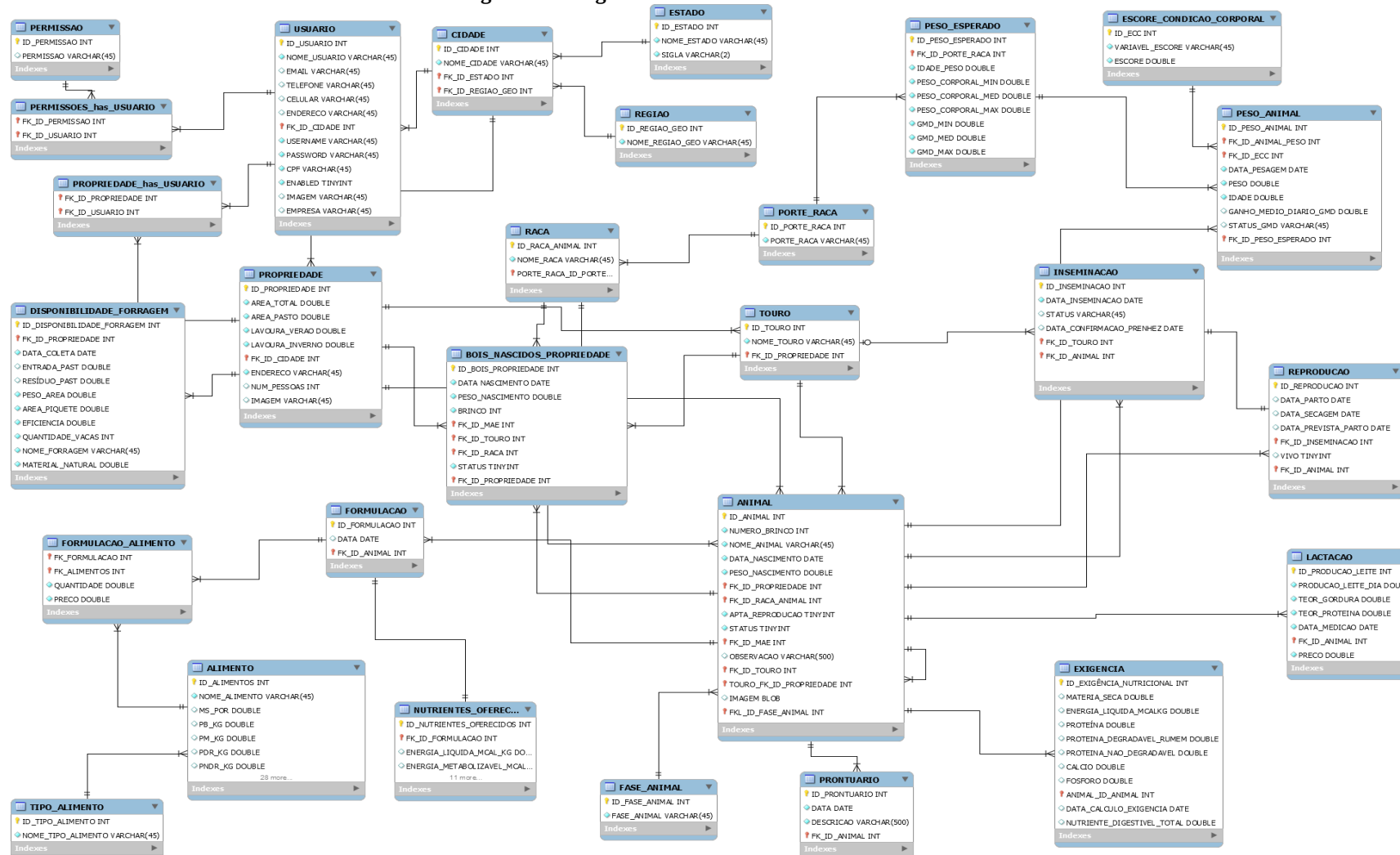
Caso de Uso: UC2 Escolher alimentos
Atores: Técnico
Precondições: Cálculo da exigência nutricional de cada animal deve ter sido realizado pelo sistema, na etapa anterior a escolha.
Pós-condições: Relatório da quantidade de alimento que será fornecida para cada animal.
Sequência típica de eventos (Fluxo Principal): Esse caso de uso inicia quando: 1. [IN] Ator seleciona os alimentos que farão parte do balanceamento. 2. [IN] Ator clica no botão “Iniciar balanceamento”. 3. [OUT] O sistema verifica para quais animais foi calculada a exigência nutricional. 4. [IN] O usuário insere a quantidade de alimento necessário. 5. [OUT] O sistema calcula os nutrientes oferecidos. 6. [IN] Usuário compara os valores exigência x oferecido. 7. [IN] Usuário conclui balanceamento. 8. [OUT] O caso de uso é finalizado

Fonte: Autoria própria.

4.2.3 Diagrama Entidade-Relacionamento

A Figura 3 apresenta o diagrama de entidades e relacionamentos do banco de dados.

Figura 3 – Diagrama Entidade-Relacionamento



Fonte: Autoria própria

A entidade “propriedade” armazena os dados que caracterizam o uso da área, a quantidade de pessoas que trabalham na atividade de bovinocultura do leite e sua localização, que inclui o relacionamento com a entidade “cidade”, que está relacionada com as entidades “estado” e “regiao_geografica”. Além disso, a propriedade possui uma entidade “disponibilidade_forragem”, na qual são cadastrados os dados das pastagens, como a quantidade de animais que pastejam por piquete.

O cadastro dos bovinos é feito na entidade “animais” e “bois_propriedade”. Nessa última serão cadastrados apenas os bois que nascem na propriedade, por não fazerem parte do processo de balanceamento, mas precisarem do registro. Os demais serão cadastrados em “animais”, com dados iniciais de peso, brinco, filiação, data de nascimento entre outros atributos.

A entidade “animais” se relaciona com quatro linhas de dados: peso, reprodução, produção e alimentação. O cadastro do peso, “peso_animal”, visa acompanhar a evolução do ganho de peso e se está dentro dos parâmetros estabelecidos, que constam na entidade “peso_espeado”. As entidades “reproducao” e “inseminacao” fazem o registro das datas de parto, secagem e inseminação que são dados necessários para definir em que fase o animal se encontra. Isso interfere na alimentação que será oferecida. A entidade “produção” cadastra os dados da quantidade de leite produzido por cada vaca e sua composição, de forma a acompanhar a evolução no decorrer do período de lactação.

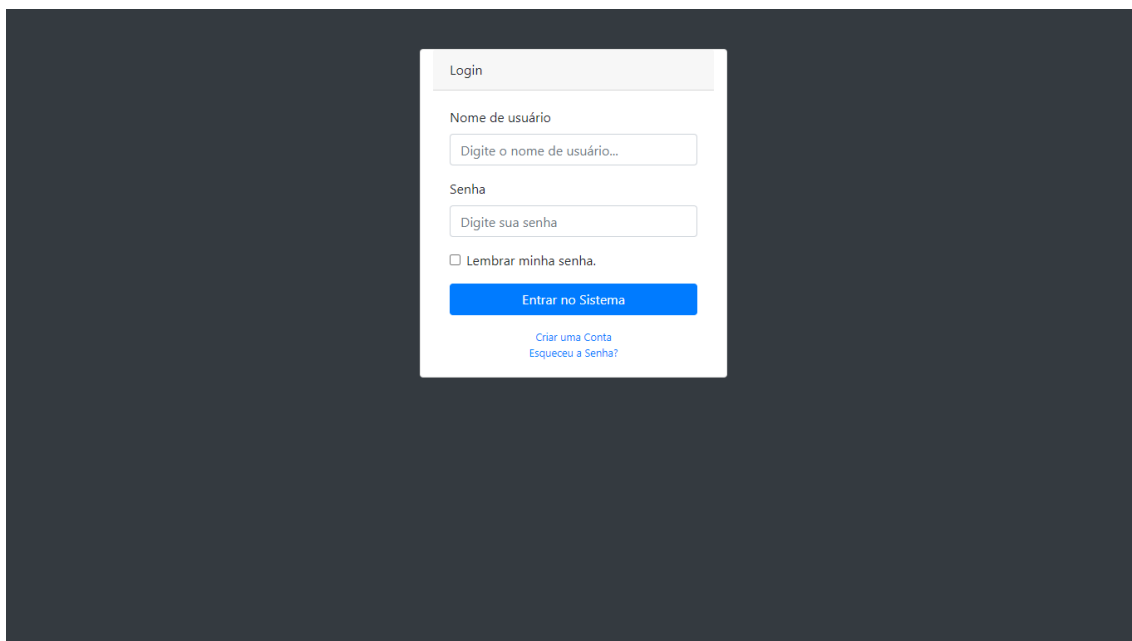
As entidades relacionadas à alimentação farão o balanceamento nutricional de cada animal. Os primeiros cálculos serão registrados em “exigencia_nutricional”, entidade que possui como atributos as quantidades de nutrientes que cada vaca necessitará, para que, além de se manter, possa aumentar a sua produção de leite. Na entidade “alimentos” será registrada a composição química dos mais variados alimentos, que serão utilizados na entidade “formulacao” para calcular a quantidade dos alimentos, indicados pelo pesquisador, que serão necessários.

4.3 APRESENTAÇÃO DO SISTEMA

As telas do sistema Web foram desenvolvidas com o *framework Bootstrap* e com o *Thymeleaf*, mecanismo que modela a camada de visualização da aplicação MVC.

O acesso ao sistema é feito pela tela de autenticação, na qual o usuário insere o *username* e a senha cadastrada (Figura 4). Caso não tenha acesso, pode acessar a tela de cadastro e inserir os dados necessários (Figura 5). O novo usuário terá acesso com permissão de técnico.

Figura 4 – Tela de Login

A imagem mostra a tela de login do sistema, centralizada em um fundo escuro cinza. O formulário de login é uma caixa branca com o título "Login" em um cabeçalho cinza claro. Abaixo do título, há o rótulo "Nome de usuário" seguido de um campo de entrada com o placeholder "Digite o nome de usuário...". Logo abaixo, há o rótulo "Senha" seguido de um campo de entrada com o placeholder "Digite sua senha". Abaixo dos campos, há uma opção "Lembrar minha senha." com uma caixa de seleção desmarcada. No final do formulário, há um botão azul com o texto "Entrar no Sistema". Abaixo do botão, há dois links azuis: "Criar uma Conta" e "Esqueceu a Senha?".

Fonte: Autoria própria

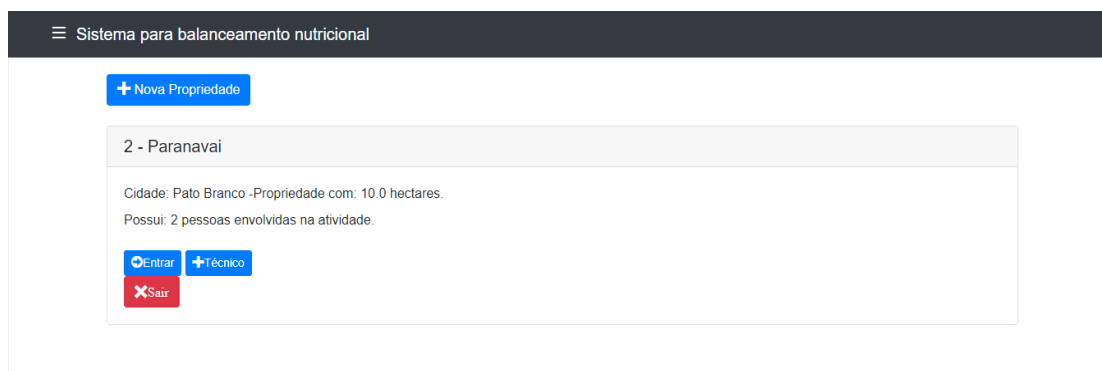
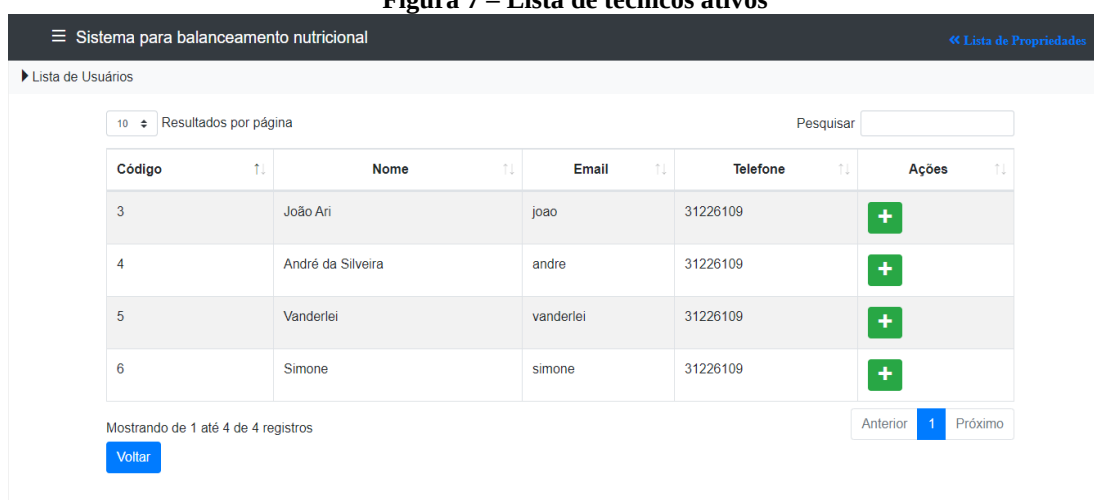
Figura 5 – Cadastro de usuário

Nome Completo	Nome de usuário
Nome completo	Nome de usuário para acesso ao sistema
E-mail	CPF
	Informe seu CPF
Senha	Confirme a senha
Senha	Confirme sua senha
Telefone	Empresa
Telefone	Empresa em que trabalha
Celular	Endereço
Celular	Endereço
Cidade	
Selecione...	
Salvar	

Fonte: Autoria própria

Após realizar o *login*, o usuário com permissão de técnico ou proprietário, visualiza a lista de propriedades à qual está vinculado e que pode fazer o acompanhamento do rebanho, conforme apresentado na Figura 6.

Cada propriedade é apresentada em forma de *card*, com a opção para acessar os dados do rebanho específico daquela propriedade, através do botão “Entrar”. O botão “Técnico”, apresenta a lista de usuários com a permissão de técnico que estão ativos e que ainda não estão vinculados à propriedade, mas que podem ser adicionados à propriedade referenciada (Figura 7). Se o usuário clicar no botão “Sair”, poderá desvincular-se da propriedade e o *card* não será mais listado.

Figura 6 – Lista de propriedades

Fonte: Autoria própria**Figura 7 – Lista de técnicos ativos**


Código	Nome	Email	Telefone	Ações
3	João Ari	joao	31226109	+
4	André da Silveira	andre	31226109	+
5	Vanderlei	vanderlei	31226109	+
6	Simone	simone	31226109	+

Fonte: Autoria própria

O botão “Nova Propriedade” (Figura 6), fica visível apenas para os usuários que tem a permissão de técnico. Após clicar nesse botão é apresentada a tela de cadastro de propriedade e do proprietário, apresentada na Figura 8. Nessa tela, os dados da propriedade e do proprietário dela devem ser inseridos. Quando os dados são registrados, o técnico que está realizando o cadastro é vinculado à propriedade e o acesso com a permissão de proprietário é gerado.

Figura 8 – Cadastro de propriedade e usuário

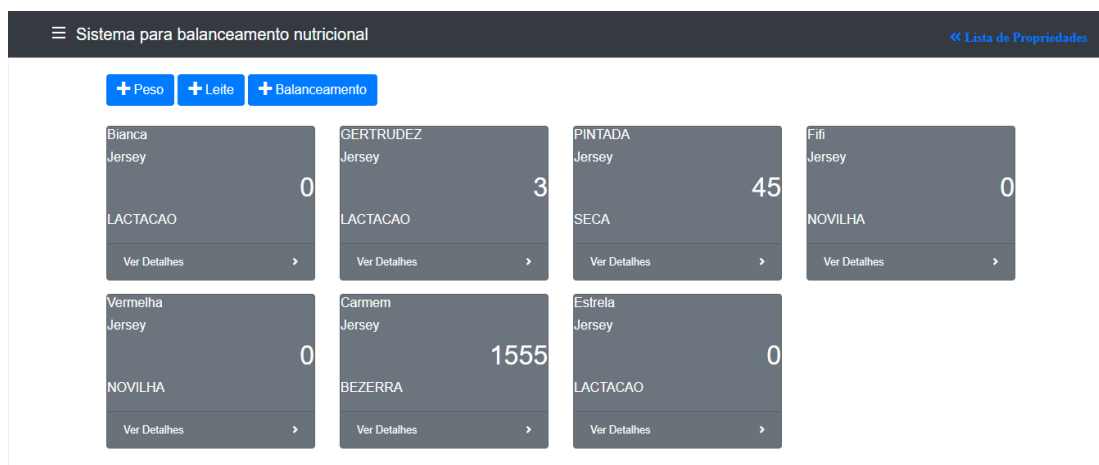
Cadastro de Nova Propriedade			
Nome da Propriedade	Área Total (hectares)	Área de Pasto (hectares)	Lavoura de Verão (hectares)
Nome da propriedade	Área total da propriedade	Área plantada com pasto	Cultura plantada no verão
Lavoura de Inverno (hectares)	Cidade		
Cultura plantada no inverno	Selecione...		
Endereço	Número de pessoas		
Endereço da propriedade (comunidade, rodovia, nº, bairro...)	Pessoas que trabalham na propriedade		

Cadastro do Proprietário	
Nome completo	Nome de usuário
Nome completo do proprietário	Nome de usuário para acesso ao sistema do proprietário
E-mail	CPF
	Informe seu CPF
Senha	Confirme a senha
Senha	Confirme sua senha
Telefone	Celular
Telefone	Celular
Endereço	Cidade
Endereço da propriedade	Selecione...

Fonte: Autoria própria

Ao acessar a propriedade, o usuário autenticado tem acesso à lista de animais pertencentes àquela propriedade, em forma de *cards*. Esses elementos apresentam os dados referentes ao nome do animal, raça, brinco e fase de produção. O *link* “Ver Detalhes”, quando clicado, apresenta todos os detalhes do animal referenciado, conforme apresentado na Figura 9.

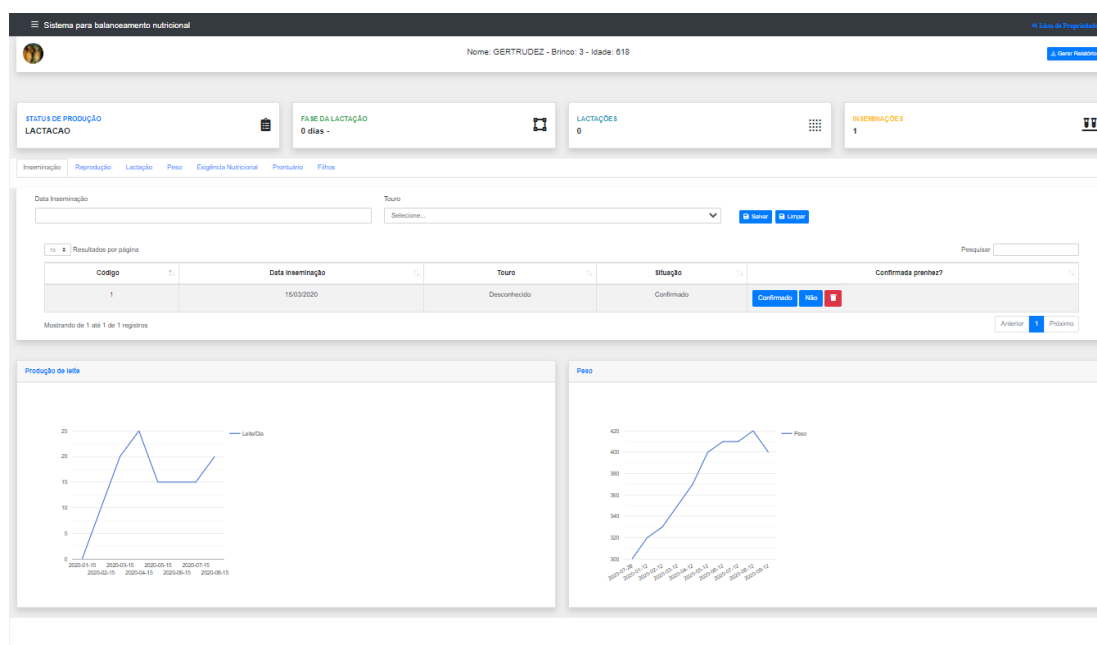
Figura 9 – Lista de animais da propriedade



Fonte: Autoria própria

Após clicar no link “Ver Detalhes” do card, a tela apresentada na Figura 10, exibe os detalhes de cada animal pertencente à propriedade. Após a identificação do animal, há quatro *cards*, apresentados horizontalmente, que trazem dados relacionados a fase que o animal se encontra, há quantos dias está em lactação, a quantidade de vezes que o animal entrou em lactação e a quantidade de vezes que foi inseminada.

Figura 10 – Detalhes dos animais



Fonte: Autoria própria

A primeira aba é denominada “Inseminação”, apresenta o cadastro das inseminações do animal. O usuário pode informar que foi realizada uma nova inseminação. Caso a prenhez não seja confirmada (botão “Não” da Figura 11), a

situação é alterada para “não prenha”, o que permite realizar nova inserção de uma inseminação. Se a prenhez for confirmada pelo médico veterinário no ultrassom, é registrado por meio do botão “Confirmado” alterando a situação do animal, conforme apresentado na Figura 11. Quando essa sequência é realizada, um novo registro da aba reprodução é gerado.

Figura 11 – Cadastro de inseminação

Sistema para balanceamento nutricional « Lista de Propriedades

Nome: Estrela - Brinco: 0 - Idade: 618 Gerar Relatório

STATUS DE PRODUÇÃO LACTAÇÃO **FASE DA LACTAÇÃO** 0 dias **LACTAÇÕES** 3 **INSEMINAÇÕES** 4

Inseminação Reprodução Lactação Peso Exigência Nutricional Prontuário Filhos

Data Inseminação Touro Salvar Limpar

10 Resultados por página Pesquisar

Código	Data Inseminação	Touro	Situação	Confirmada prenhez?
3	14/09/2020	Legal	Não Prenha	Confirmado Não ❌
4	04/03/2019	Desconhecido	Confirmado	Confirmado Não ❌
5	09/11/2020	Legal	Confirmado	Confirmado Não ❌
6	02/11/2020	Desconhecido	Confirmado	Confirmado Não ❌
7	12/11/2020	Legal	Confirmado	Confirmado Não ❌

Mostrando de 1 até 5 de 5 registros Anterior 1 Próximo

Fonte: Autoria própria

Na Figura 12 pode-se observar a lista de reproduções. O sistema calcula a data prevista do parto e a data da secagem, para que o usuário possa acompanhar o desenvolvimento da gestação e, no momento certo, esgotar o animal e prepará-lo para o período de pré-parto.

Figura 12 – Cadastro de reprodução

Sistema para balanceamento nutricional « Lista de Propriedades

Nome: Estrela - Brinco: 0 - Idade: 618 Gerar Relatório

STATUS DE PRODUÇÃO LACTAÇÃO FASE DA LACTAÇÃO 0 dias - LACTAÇÕES 3 INSEMINAÇÕES 4

Inseminação Reprodução Lactação Peso Exigência Nutricional Prontuário Filhos

10 Resultados por página Pesquisar

Código	Data Prevista do Parto	Data Secagem	Data do Parto	Vivo?	Ações
2	12/12/2019	13/10/2019	09/11/2020	Sim	Incluir Parto
3	19/08/2021	20/06/2021	09/11/2020	Não	Incluir Parto
4	12/08/2021	13/06/2021	09/11/2020	Sim	Incluir Parto
5	22/08/2021	23/06/2021		Aguardando parto	Incluir Parto

Mostrando de 1 até 4 de 4 registros Anterior 1 Próximo

Fonte: Autoria própria

Quando o parto efetivamente ocorre, a inclusão é feita por meio do botão “Incluir Parto” apresentado na Figura 12, gerando um novo registro, conforme a situação do parto. Nas Figuras 13 e 14, pode-se observar a sequência do cadastro para um bezerro fêmea, que nasceu viva.

Figura 13 – Escolha da situação do nascimento

Sistema para balanceamento nutricional « Lista de Propriedades

Cadastrar Nascimento Listar Animais

Em qual situação o bezerro(a) nasceu?

☒ Vivo ☐ Morto

Qual o sexo do bezerro que nasceu?

☒ Fêmea ☐ Macho

Fonte: Autoria própria

Figura 14 – Cadastro de bezerro fêmea

Sistema para balanceamento nutricional « Lista de Propriedades

Cadastrar Nascimento Listar Animais

Em qual situação o bezerro(a) nasceu?

☒ Vivo ☐ Morto

Nome do Animal Número do Brinco Peso no nascimento

Data Nascimento Raça

Salvar

Fonte: Autoria própria

Quando a sequência é Vivo-Fêmea o registro é salvo na tabela “Animais”. Se a sequência for Vivo-Macho será feita a inserção na tabela de “Bois Nascido na Propriedade”. Selecionado o *checkbox* “Morto”, a inserção será registrada na tabela “Prontuário” do animal. Em todos os casos é inserido o registro da data efetiva do parto na reprodução.

Nas abas “Lactação”, “Peso” e “Prontuário” é realizada a inclusão, exclusão, listagem e edição dos dados de produção de leite, do peso apresentado pelo animal e inseridos registros de vacinação, doenças, medicação entre outras informações pertinentes à saúde do animal no prontuário. A interface para realizar essas operações segue o padrão apresentado na Figura 15.

Na tabela que apresenta a listagem dos resultados, a coluna “ações” possui dois botões: o botão azul permite alterar um registro e o botão vermelho permite excluir o registro.

Figura 15 – Cadastro de reprodução

Figura 15 – Cadastro de reprodução

Sistema para balanceamento nutricional

« Lista de Propriedades

STATUS DE PRODUÇÃO
LACTACAO

FASE DA LACTAÇÃO
1 dias - Início

LACTAÇÕES
1

INSEMINAÇÕES
1

Inseminação Reprodução Lactação **Peso** Exigência Nutricional Prontuário Filhos

Data Medição:
 Produção de leite do dia: Quantos Kg de leite tirado
 Teor de Gordura %: Teor de gordura em %
 Teor de Proteína %: Teor de proteína em %
 R\$/L de Leite: Preço do litro de leite

10 Resultados por página

Pesquisar

Código	Data Medição	Produção de leite	Teor de Gordura	Teor de Proteína	R\$/Litro	Ações
1	15/01/2020	0.0	5.0	5.0	1.2	Editar Excluir
2	15/02/2020	10.0	5.0	5.0	1.2	Editar Excluir
3	15/03/2020	20.0	5.0	5.0	1.2	Editar Excluir
4	15/04/2020	25.0	5.0	5.0	1.2	Editar Excluir

Fonte: Autoria própria

As abas “Exigência Nutricional” e “Filhos” listam os registros conforme apresentado na Figura 16.

Figura 16 – Lista de filhos

Código	Brinco	Nome	Data de nascimento	Fase	Ações
11	1555	Carmem	09/11/2020	BEZERRA	

Fonte: Autoria própria

Na tela principal da propriedade (apresentada na Figura 9) tem os botões adicionar Peso e adicionar Lactação. São telas que tem como objetivo facilitar a inserção dos dados, sem necessidade de entrar no *card* de “Detalhes” de cada animal. Na primeira são listados todos os animais da propriedade, independente da fase em que se encontra. Já na tela de “Lactações” é listado apenas os animais em que a fase seja igual a “lactação”, pois são inseridos os dados de produção de leite/dia, dentre outros dados, seguindo o padrão apresentado na Figura 17.

Figura 17 – Cadastro de pesos ou lactações do rebanho

#	Brinco	Nome	Produção de Leite	% Teor de Gordura	% Teor de Proteína	Data Medição	R\$/L
1	0	DESCONHECIDA					
2	3	GERTRUDEZ					

Fonte: Autoria própria

O terceiro botão da tela principal adicionar “Balanceamento” (apresentado na Figura 9), visível apenas para os usuários com a permissão de técnico, permite iniciar o processo de balanceamento nutricional. Ao clicar nesse botão, o usuário é direcionado para a primeira etapa, que é realizar o cálculo da exigência nutricional dos animais. Conforme apresentado na Figura 18 são listados os animais que estejam na fase de lactação, seca ou pré-parto.

Figura 18 – Cadastro para cálculo das exigências nutricionais

Id	Brinco	Nome	Temperatura °C	Perda de peso	Ganho de peso
1	0	DESCONHECIDA			
2	3	GERTRUDEZ			
3	45	PINTADA			

Fonte: Autoria própria

Após informar os dados e clicar no botão para salvar o registro, o sistema realiza os cálculos necessários e armazena a exigência nutricional de cada animal da lista. Esses dados serão utilizados posteriormente na comparação com a quantidade de nutrientes oferecidos.

Na segunda etapa do processo, demonstrado na Figura 19, o usuário irá escolher os alimentos que serão fornecidos para o rebanho, informando o custo do produto. Após escolhida a lista de alimentos, pode-se iniciar o balanceamento.

Figura 19 – Escolha dos alimentos

id	Alimento	Valor	Quantidade	Excluir
6	Aveia 01, Primeiro Pastejo -VOLUMOSO	8	null	☐
60	Gérmen Zaeli -CONCENTRADO	7	null	☐

Fonte: Autoria própria

Na última etapa do processo, o sistema lista os animais que estão na fase de lactação, seca ou pré-parto, em forma de *carousel*. Em cada *slide* são listadas duas tabelas, a primeira apresenta a exigência nutricional que foi calculada para o animal. Na segunda tabela estão os nutrientes oferecidos, de acordo com a quantidade informada nos *inputs* do final da tela, conforme apresentado na Figura 20.

Figura 20 – Tela para incluir a quantidade de alimento

Sistema para balanceamento nutricional

« Lista de Propriedades

Exigência nutricional

Código	Data	IMS (kg/dia)	EL (Mcal/Kg)	NGT (Kg/dia)	PM (Kg/dia)	PB (Kg/dia)	PDR Kg	PNDR Kg	Ca (Kg/dia)	P (Kg/dia)
5	2020-10-31	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Nutrientes oferecidos

FDN	EE	FDNe	EM	IMS (kg/dia)	EL (Mcal/Kg)	NGT (Kg/dia)	PM (Kg/dia)	PB (Kg/dia)	PDR Kg	PNDR Kg	Ca (Kg/dia)	P (Kg/dia)
0.33	0.31	0.00	1.22	3.21	0.77	2.32	0.25	0.38	0.23	0.14	0.00	0.01

Concluir

Aveia 01, Primeiro Pastejo

4

Gérmen Zaeli

3

Fonte: Autoria própria

Quando os valores da tabela “Nutrientes Oferecidos” estiverem próximos ou iguais aos valores da tabela “Exigência Nutricional”, o balanceamento está pronto, podendo ser concluído, a critério do usuário.

Na Figura 21, pode-se observar o menu lateral, com o item “Rebanho”, no qual o usuário tem acesso ao relatório de formulações realizadas, poderá incluir a compra de um animal, editar os dados de um animal cadastrado e visualizar a lista de bois que nasceram na propriedade.

O item “Propriedade” dá acesso ao cadastro da propriedade, que pode ser editado, ao cadastro de forragens disponíveis na propriedade e de touros utilizados nas inseminações.

Figura 21 – Menu lateral

The interface shows a lateral menu on the left with the following items: Home, Rebanho, Formulações realizadas, Listar Rebanho, Incluir Compra de Animal, Bois Nascidos, and Propriedade. The main area displays a grid of cards for different animals and their nutritional status. The cards are organized into two rows and four columns. The first row contains cards for Bianca Jersey (LACTACAO, 0), GERTRUDEZ Jersey (LACTACAO, 3), PINTADA Jersey (SECA, 45), and Fifi Jersey (NOVILHA, 0). The second row contains cards for Vermelha Jersey (NOVILHA, 0), Carmem Jersey (BEZERRA, 1555), and Estrela Jersey (LACTACAO, 0). Each card has a 'Ver Detalhes' button.

Fonte: Autoria própria

A Figura 22 apresenta as formulações realizados, agrupadas por data e a Figura 23 a visualização do relatório com os balanceamentos feitos.

Figura 22 – Formulações agrupadas por data

The interface shows a table titled 'Lista de Formulações Realizadas'. The table has three columns: 'Quantidade de Balanceamentos', 'Data Formulação', and 'Ação'. The table contains three rows of data. The first row shows 1 balanceamento on 18/06/2020. The second row shows 2 balanceamentos on 18/05/2020. The third row shows 4 balanceamentos on 09/11/2020. Each row has a yellow eye icon and a blue download icon in the 'Ação' column. The interface also includes a search bar, a dropdown for 'Resultados por página' (set to 10), and pagination controls (Anterior, 1, Próximo).

Quantidade de Balanceamentos	Data Formulação	Ação
1	18/06/2020	
2	18/05/2020	
4	09/11/2020	

Fonte: Autoria própria.

Figura 23 – Relatório do balanceamento

2020-11-09 1 / 1

BALANCEAMENTO NUTRICIONAL DAS VACAS EM LACTAÇÃO

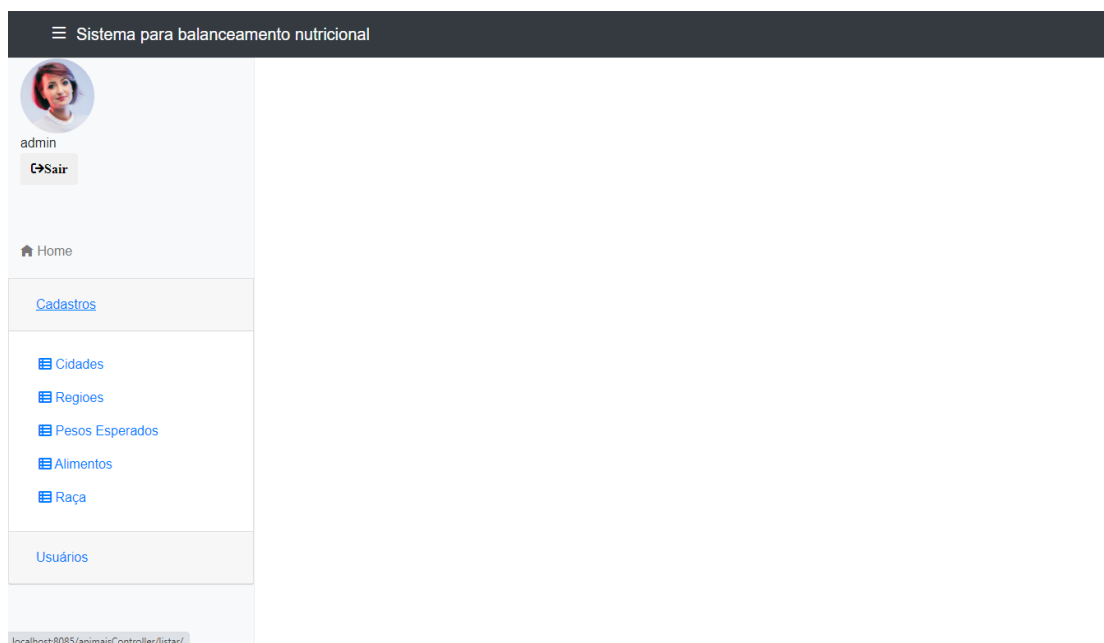
Data Formulação 09/11/20 00:00

Nome	Brinco	Alimento	Quantidade
Bianca	0	Aveia 01, Primeiro	5.0
Bianca	0	Pastejo	2.0
Bianca	0	casca	0.8
GERTRUDEZ	3	Aveia 01, Primeiro	2.3
GERTRUDEZ	3	Pastejo	1.2
GERTRUDEZ	3	casca	0.5
PINTADA	45	Aveia 01, Primeiro	10.0
PINTADA	45	Pastejo	5.0
PINTADA	45	casca	0.9
Estrela	0	Aveia 01, Primeiro	4.0
Estrela	0	Pastejo	6.0
Estrela	0	casca	0.8

09 Novembro, 2020 Final do Relatório Página 1 de 1

Fonte: Autoria própria

O acesso com a permissão de administrador, tem como finalidade gerenciar alguns cadastros gerais, que serão usados por todas as propriedades. Sendo acessado pelo usuário por meio do menu lateral, conforme apresentado na Figura 24.

Figura 24 – Menu lateral do administrador**Fonte: Autoria própria**

Os cadastros seguem o mesmo padrão apresentado na Figura 25, com a listagem dos registros existentes no banco de dados, e possui as opções de editar, excluir e inserir um novo registro.

Figura 25 – Listagem dos pesos esperados

Sistema para balanceamento nutricional									
« Lista de Propriedades									
► Lista de Pesos Esperados									
10 Resultados por página									
Pesquisar									
Código	Porte da Raça	Idade	Peso Corporal Mínimo	Peso Corporal Médio	Peso Corporal Máximo	GMD Mínimo	GMD Médio	GMD Máximo	Ação
1	PEQUENA	0	22.5	25.0	27.5	0.24	0.267	0.294	 
2	PEQUENA	1	22.74	25.267	27.79	0.24	0.267	0.29	 
3	PEQUENA	2	22.98	25.533	28.09	0.24	0.267	0.3	 
4	PEQUENA	3	23.22	25.8	28.38	0.24	0.267	0.29	 
5	PEQUENA	4	23.46	26.067	28.67	0.24	0.267	0.29	 
6	PEQUENA	5	23.7	26.333	28.97	0.24	0.267	0.3	 
7	PEQUENA	6	23.94	26.6	29.26	0.24	0.267	0.29	 
8	PEQUENA	7	24.18	26.867	29.55	0.24	0.267	0.29	 

Fonte: Autoria própria

Também por meio do menu lateral, é possível ter acesso à lista de todos os usuários do sistema (Figura 26), e através do botão visualizar em amarelo, acessar a tela da Figura 27, na qual o administrador pode atribuir e remover permissões aos usuários cadastrados, além de vincular ou remover o acesso às propriedades.

Figura 26 – Lista de usuários do sistema

Sistema para balanceamento nutricional					
« Lista de Propriedades					
► Lista de Usuários					
10 Resultados por página					
Pesquisar					
Código	Nome	Email	Telefone	Ativo?	Ações
1	Vanessa Lucion	vanessalucion@hotmail.com	46991180118	Ativo	
2	admin	admin@hotmail.com	4699178018	Ativo	
3	João Ari	joao@hotmail.com	4699118018	Ativo	
4	André da Silveira	andre@hotmail.com	4699119018	Ativo	
5	Vanderlei	vanderlei@hotmail.com	4699168018	Ativo	
6	Simone	simone@hotmail.com	4699132018	Inativo	
7	Irma Chicoski	irma@hotmail.com	4699118018	Inativo	

Mostrando de 1 até 7 de 7 registros

Anterior 1 Próximo

[Voltar](#)

Fonte: Autoria própria

Figura 27 – Tela de permissões do usuário

Sistema para balanceamento nutricional [« Lista de Propriedades](#)

► Permissões do Usuário

Nome: Vanessa Lucion - Cidade: Pato Branco

Propriedade
 +

☒ Cadastro Ativo

Propriedades vinculadas ao usuário

Código	Propriedade	Cidade	Região	Ações
2	Paranavai	Pato Branco	Noroeste Paranaense	

Permissões
 +

Permissões do Usuario

Código	Permissão	Ações
6	ROLE_TECNICO	

[Voltar](#)

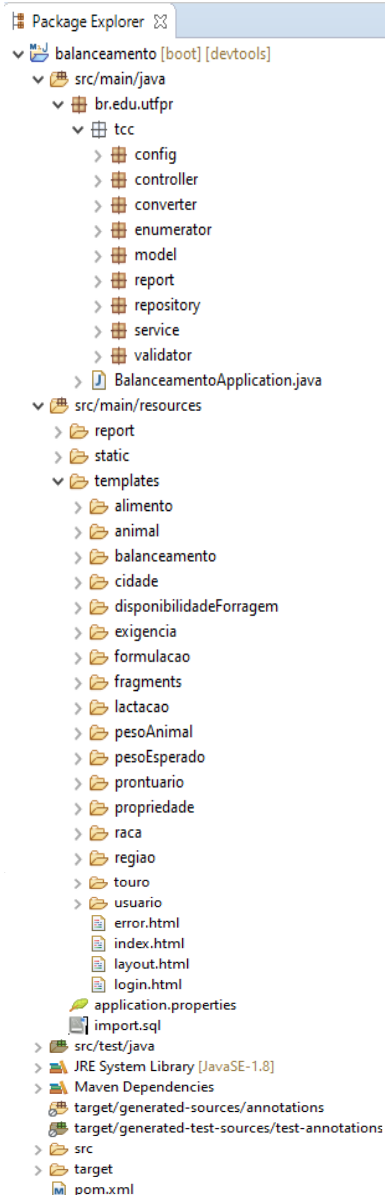
Fonte: Autoria própria

4.4. IMPLEMENTAÇÃO DO SISTEMA

O sistema foi implementado no formato *server-side*, no qual as operações são realizadas no servidor e utilizado o padrão *Model-View-Controller* (MVC). Esse padrão permite a separação do código em três camadas: a *model* faz a manipulação lógica dos dados por meio das classes de objetos criados e que irão gerar as tabelas do banco de dados, por meio do Hiberbate. A *view* é a camada que o usuário faz a interação com o sistema e a *controller* faz a ligação entre as duas primeiras camadas. Para gerenciar as dependências do projeto foi utilizado o *Apache Maven*.

A Figura 26 apresenta a estrutura do *back-end*, desenvolvido utilizando as tecnologias Java e Spring e a estrutura dos arquivos do *front-end*, que foi utilizado o HTML5, Bootstrap, CSS e Thymeleaf.

Figura 28 – Estrutura do projeto



Fonte: Autoria própria

Para manipular os dados de alimentos, cidade, forragens disponíveis na propriedade, os pesos esperados, por padrão, dos animais, o prontuário, além dos registros das raças e das regiões, segue a mesma estrutura do *Controller* apresentado na Listagem 1, na qual é realizado as operações de excluir, listar, salvar e editar.

Listagem 1 – Controller das raças

```
@Controller
@RequestMapping("/racas")
public class RacaController {

    @Autowired
    private RacaRepository racaRepository;
```

```

    @DeleteMapping("/{id}")
    public String remover(@PathVariable Long id, RedirectAttributes
attributes) {
        racaRepository.deleteById(id);
        attributes.addFlashAttribute("mensagem", "Registro removido com
sucesso!");

        return "redirect:/racas";
    }

    @GetMapping
    public ModelAndView listar() {
        ModelAndView modelAndView = new ModelAndView("raca/lista");
        modelAndView.addObject("racas", racaRepository.findAll());
        return modelAndView;
    }

    @GetMapping("/novo")
    public ModelAndView novo(Raca raca) {
        ModelAndView modelAndView = new ModelAndView("raca/cadastro");
        modelAndView.addObject(raca);
        modelAndView.addObject("porteList", PorteRaca.values());
        return modelAndView;
    }

    @PostMapping("/")
    public ModelAndView salvar(@Valid Raca raca, BindingResult result,
RedirectAttributes attributes) {
        if (result.hasErrors()) {
            return novo(raca);
        }
        racaRepository.save(raca);
        attributes.addFlashAttribute("mensagem", "Registro salvo com
sucesso!");
        return new ModelAndView("redirect:/racas");
    }

    @GetMapping("/{id}")
    public ModelAndView editar(@PathVariable Long id) {
        return novo(racaRepository.getOne(id));
    }
}

```

Fonte: Autoria própria.

A segurança de acesso ao sistema com autenticação, proteção e armazenamento de senhas é feita pela biblioteca Spring Security. A Listagem 2 apresenta a configuração utilizada.

Listagem 2 – Classe de configuração Spring Security

```

@EnableWebSecurity
@EnableGlobalMethodSecurity(securedEnabled = true)
public class WebSecurityConfig extends WebSecurityConfigurerAdapter{

```



```

@Autowired
private UsuarioService usuarioService;

@Override
protected void configure(HttpSecurity http) throws Exception {
    http.csrf().disable()
        .exceptionHandling().accessDeniedPage("/403")
        .and().formLogin().loginPage("/login")
        .defaultSuccessUrl("/propriedades", true)
        .failureUrl("/home/loginError").permitAll()
        .and().logout()
        .logoutSuccessUrl("/login")
        .and().exceptionHandling().accessDeniedPage
("/home/acessoNegado")
        .and().authorizeRequests()

        //ACESSO PRIVADO TÉCNICO
        .antMatchers("/usuarios/listarTecnico/**")
        .hasAnyRole("TECNICO")
        .antMatchers("/exigenciaController/**").hasRole("TECNICO")
        .antMatchers("/formulaçãoAlimentoController/**").hasRole("TECNICO")
        .antMatchers("/formulaçãoController/**").hasRole("TECNICO")
        .antMatchers("/nutrientesOferecidosController/**").hasRole("TECNICO")

//ACESSOS PRIVADOS DO ADMIN
        .antMatchers("/usuarios/**").hasAnyRole("ADMIN")
        .antMatchers("/racas/**").hasRole("ADMIN")
        .antMatchers("/cidades/**").hasRole("ADMIN")
        .antMatchers("/pesosEsperados/**").hasRole("ADMIN")
        .antMatchers("/regioes/**").hasRole("ADMIN")
        .antMatchers("/pesosEsperados/**").hasRole("ADMIN")
        .antMatchers("/alimentoController/**").hasRole("ADMIN")

//ACESSOS PRIVADOS TÉCNICO E PROPRIETÁRIO
        .antMatchers("/animaisController/**")
        .hasAnyRole("TECNICO", "PROPRIETARIO")
        .antMatchers("/boisPropriedadeController/**").hasAnyRole("TECNICO",
"PROPRIETARIO")
        .antMatchers("/inseminacaoController/**").hasAnyRole("TECNICO",
"PROPRIETARIO")
        .antMatchers("/lactacaoController/**")
        .hasAnyRole("TECNICO", "PROPRIETARIO")
        .antMatchers("/pesosAnimais/**").hasAnyRole("TECNICO", "PROPRIETARIO")
        .antMatchers("/prontuarioController/**").hasAnyRole("TECNICO",
"PROPRIETARIO")
        .antMatchers("/reproducaoController/**").hasAnyRole("TECNICO",
"PROPRIETARIO")
        .antMatchers("/touroController/**")
        .hasAnyRole("TECNICO", "PROPRIETARIO")
        .antMatchers("/reportController/animais/visualizar/**")
        .hasAnyRole("TECNICO",
"PROPRIETARIO")
        .antMatchers("/formulaçãoController/listar/**")
        .hasAnyRole("TECNICO", "PROPRIETARIO")
        .antMatchers("/**").authenticated();
    }
}

```

```

//LIBERAR ACESSO AOS ARQUIVOS ESTÁTICOS
@Override
public void configure(WebSecurity web) throws Exception {
    web.ignoring()
        .antMatchers("/layout/**")
        .antMatchers("/images/**")
        .antMatchers("/webjars/**")
        .antMatchers("/static/**")
        .antMatchers("/usuarios/");
}

@Bean
@Override
protected UserDetailsService userDetailsService() {
    return usuarioService;
}

@Bean
protected PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Override
protected void configure(AuthenticationManagerBuilder auth)
    throws Exception {
    auth.userDetailsService( userDetailsService() )
        .passwordEncoder( passwordEncoder() );
}
}

```

Fonte: Autoria própria.

Como o relacionamento entre as tabelas “Usuário” e “Permissão” é de muitos para muitos, na classe de “Usuario” foram criados dois métodos que adicionam e removem os registros da *collection Set<>*, exibidos na Listagem 3.

Listagem 3 – Classe de usuário

```

@Entity
@AllArgsConstructor
@NoArgsConstructor
public class Usuario extends AbstractEntity<Long> implements UserDetails{

    @ManyToMany(mappedBy = "usuarios", fetch = FetchType.EAGER)
    @Setter @Getter
    private Set<Propriedade> propriedades;

    @ManyToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    @Setter @Getter
    private Set<Permissao> permissoes;

    @Override

```

```

    public Collection<? extends GrantedAuthority> getAuthorities() {
        List<GrantedAuthority> auto = new ArrayList<>();
        auto.addAll(getPermissoes());

        return auto;
    }

    public void addPermissao(Permissao permissao) {
        if (permissoes == null) {
            permissoes = new HashSet<>();
        }
        permissoes.add(permissao);
    }

    public void removePermissao(Permissao permissao) {
        if (permissoes == null) {
            permissoes = new HashSet<>();
        }
        permissoes.remove(permissao);
    }
}

```

Fonte: Autoria própria.

Da mesma forma foi feito para o objeto Propriedade, o relacionamento com “Usuario” é de muitos para muitos.

A Listagem 4 apresenta a estrutura do *controller* de usuário. O método “permissoesUsuario” lista todas as permissões e propriedades a que um usuário possui ou pertence e também se está ativo. Esses dados poderão ser alterados por meio do método atualizar, no caso do atributo *Enabled*, já para o administrador vincular uma nova propriedade ao usuário, é utilizado o método “vincularPropriedadeUsuario”. Uma nova permissão é realizado pelo método “addPermissaoUsuario”.

Listagem 4 – Método lista permissões e acessos

```

@GetMapping("/permissoesUsuario/{idUserario}")
public ModelAndView permissoesUsuario(@PathVariable Long idUsuario,
Propriedade, Permissao) {
    ModelAndView modelAndView = new ModelAndView("usuario/permissoes");
    modelAndView.addObject(propriedade);
    modelAndView.addObject(permissao);
    Usuario usuario = usuarioRepository.getOne(idUsuario);
    modelAndView.addObject("usuarios", usuarioRepository.getOne(idUsuario));
    modelAndView.addObject("permissoesUsuarioList",
usuario.getAuthorities());
    modelAndView.addObject("permissoesList",
permissaoRepository.findAll());
    modelAndView.addObject("propriedadesUsuarioList",
propriedadeRepository.findByUsuariosId(idUsuario));
    modelAndView.addObject("propriedadesList",
propriedadeRepository.findAll());
}

```

```

        return modelAndView;
    }

    @PostMapping("/usuarioAtivo/{idUserario}")
    public ResponseEntity<> atualizar(@RequestBody @Valid Usuario us,
    @PathVariable Long idUsuario, BindingResult result,
    Model model, RedirectAttributes attributes) {
        Usuario usuario = usuarioRepository.getOne(idUsuario);
        if(us.isEnabled() == true) {
            usuario.setEnabled(true);
            usuarioRepository.save(usuario);
            return new ResponseEntity<>(HttpStatus.OK);
        }else if(us.isEnabled() == false) {
            usuario.setEnabled(false);
            usuarioRepository.save(usuario);
            return new ResponseEntity<>(HttpStatus.OK);
        }else {
            return new ResponseEntity<>(result.getAllErrors(),
            HttpStatus.BAD_REQUEST);
        }
    }

    @PostMapping ("/vincular/{idUserario}")
    public ModelAndView vincularPropriedadeUsuario(@Valid Propriedade
    prop,@PathVariable Long idUsuario){
        Propriedade propriedade = propriedadeRepository.getOne(prop.getId());
        propriedade.addUsuario(getUsuario(idUsuario));
        propriedadeRepository.save(propriedade);
        return new ModelAndView("redirect:/usuarios/permissoesUsuario/" +
        idUsuario);
    }

    @PostMapping ("/addPermissao/{idUserario}")
    public ModelAndView addPermissaoUsuario(@Valid Permissao per, @PathVariable
    Long idUsuario){
        Usuario usuario = usuarioRepository.getOne(idUsuario);
        usuario.addPermissao(getPermissao(per.getId()));
        usuarioRepository.save(usuario);
        return new ModelAndView("redirect:/usuarios/permissoesUsuario/" +
        idUsuario);
    }

```

Fonte: Autoria própria.

Quando um usuário autenticado, com a permissão de técnico, deseja incluir outras pessoas para acompanhar uma mesma propriedade, o método “listarTecnico”, vai listar todos os usuários com a permissão de técnico ativos, e que ainda não acompanham aquela propriedade, conforme apresentado na Listagem 5.

Listagem 5 – Lista de técnicos ativos

```

@GetMapping ("/listarTecnico/{id}")
public ModelAndView listarTecnico(@PathVariable Long id) {

```

```

        ModelAndView modelAndView = new ModelAndView("usuario/lista");
        List<Usuario> listTecnicosAtivos =
usuarioRepository.findByUsuarioAtivoRoleTecnico("ROLE_TECNICO", "S");
        List<Usuario> listUsuarioVinculadoPropriedade =
usuarioRepository.findByUsuarioVinculadoPropriedade(id);
        List<Usuario> listUsuarios = new ArrayList<Usuario>();
        for(Usuario usuario :listTecnicosAtivos) {
            if(!seContem(usuario, listUsuarioVinculadoPropriedade)){
                listUsuarios.add(usuario);
            }
        }

        modelAndView.addObject("tecnicosList", listUsuarios);
        return modelAndView;
    }

    public boolean seContem(Usuario item, List<Usuario> lista) {
        boolean contem = false;
        for(Usuario u : lista) {
            if(u.getId() == item.getId()) {
                contem = true;
                break;
            }
        }
        return contem;
    }
}

```

Fonte: Autoria própria.

Os métodos “verificarUsername” e “verificarCPF”, da Listagem 6, são utilizados durante o cadastro inicial do usuário e da propriedade, verificando no banco de dados se o valor preenchido já está cadastrado ou se é válido, retornando mensagem para o lado cliente.

Para inserir um novo registro no banco de dados é utilizado o método “salvar”, os dados salvos são apenas do cadastro inicial do usuário. O que vai tornar único um usuário no sistema, é o CPF e o *username*.

Listagem 6 – Salva novo cadastro de usuário

```

@PostMapping("/")
public String salvar(@Valid Usuario usuario, BindingResult result,
    Model model, RedirectAttributes attributes) {
    String pass = usuario.getPassword();
    String encodedPassword = usuario.getEncodedPassword(pass);

    usuario.setPassword(encodedPassword);
    usuario.addPermissao(getPermissao());
    usuario.setEnabled(true);

    try {

```

```

        usuarioRepository.save(usuario);
    }catch(DataIntegrityViolationException ex) {
        result.reject("username", "Ops... Nome do usuário ou CPF
já existem!!!");
        return "usuario/cadastro";
    }
    return "redirect:/usuarios/cadastroRealizado";
}

@PostMapping ("/verificarCPF")
private ResponseEntity<String> verificarCPF(@RequestBody @Valid
Usuario usuario, BindingResult result, Model model, RedirectAttributes
attributes) {
    if(!ValidarCPF.isCPF(usuario.getCpf())) {
        return new ResponseEntity<>("Informe um CPF válido!",
HttpStatus.OK);
    }else {
        List<Usuario> usuarioList =
usuarioRepository.findByCpfUsuario(usuario.getCpf());

        if(usuarioList.size() > 0) {
            return new ResponseEntity<>("Já existe um usuario
cadastrado com este cpf", HttpStatus.OK);
        }else {
            return new ResponseEntity<>("CPF válido!!!",
HttpStatus.OK);
        }
    }
}

@PostMapping ("/verificarUsername")
private ResponseEntity<String> verificarUsername(@RequestBody @Valid
Usuario usuario, BindingResult result, Model model, RedirectAttributes
attributes){
    List<Usuario> usuarioList =
usuarioRepository.findByUsernameUsuario(usuario.getUsername());

    if(usuarioList.size() > 0) {

        return new ResponseEntity<>("Já existe um usuario
cadastrado com este nome", HttpStatus.OK) ;
    }else {
        return new ResponseEntity<>("Usuário válido!!!",
HttpStatus.OK);
    }
}
}

```

Fonte: Autoria própria.

O *Controller* das propriedades é responsável por lista e salvar os registros, bem como dar acesso aos usuários nas propriedades escolhidas. Na Listagem 7, o método “salvar”, recebe uma propriedade e um usuário que será salvo com a permissão de proprietário. Esse cadastro é realizado pelo técnico, que informa os dados da área e

também cria um *login* de acesso ao proprietário da área. O e-mail pode ser repetido, pois quando for cadastrar um agricultor que não tenha e-mail, poderá usar o mesmo do técnico já cadastrado. Ao salvar a propriedade, já fica vinculada a ela o técnico que cadastrou e o proprietário.

Listagem 7 – Método para salvar uma propriedade

```
@PostMapping("/")
public String salvar(@Valid Propriedade propriedade, Usuario usuario,
Principal principal, BindingResult result, RedirectAttributes attributes) {
    String pass = usuario.getPassword();
    String encodedPassword = usuario.getEncodedPassword(pass);
    usuario.setPassword(encodedPassword);
    usuario.addPermissao(getPermissao());
    usuario.setEnabled(true);

    try {
        usuarioRepository.save(usuario);
        propriedade.addUsuario(getUsuario(usuario.getId()));
        propriedade.addUsuario(getUsuario(principal));
        propriedadeRepository.save(propriedade);
        attributes.addFlashAttribute("sucesso", "Registro salvo
com sucesso!");
        return "redirect:/propriedades";
    } catch (DataIntegrityViolationException ex) {
        attributes.addFlashAttribute("falha", "Nome do usuário ou
CPF já existem!!");
        return "redirect:/propriedades/novo";
    }
}
```

Fonte: Autoria própria.

A Listagem 8 apresenta dois métodos relacionados ao acesso dos usuários à propriedade. No método “vincularPropriedadeUsuario”, um técnico, cadastrado e autenticado, pode incluir outros técnicos para acompanhar a propriedade. Já no método “removerUsuario”, se algum técnico, não quiser mais visualizar uma propriedade, pode sair, se desvincular.

Listagem 8 – Método para vincular técnico a propriedade

```
@PostMapping ("/vincular/{idTecnico}/{idPropriedade}")
public ModelAndView vincularPropriedadeUsuario(@PathVariable Long
idTecnico, @PathVariable Long idPropriedade){
    Propriedade propriedade =
propriedadeRepository.getOne(idPropriedade);
    propriedade.addUsuario(getUsuario(idTecnico));
    propriedadeRepository.save(propriedade);
    return new ModelAndView("redirect:/usuarios/listarTecnico/" +
idPropriedade);
}
```

```

@PostMapping("/sairPropriedade/{idPropriedade}")
public ModelAndView removerUsuario(@PathVariable Long idPropriedade,
Principal principal, RedirectAttributes attributes) {
    Propriedade propriedade =
propriedadeRepository.getOne(idPropriedade);
    propriedade.removeUsuario((getUsuario(principal)));
    propriedadeRepository.save(propriedade);
    attributes.addFlashAttribute("mensagem", "Registro removido com
sucesso!");
    return new ModelAndView("redirect:/propriedades");
}

```

Fonte: Autoria própria.

O *controller* dos animais realiza o cadastro dos nascimentos na propriedade, tanto para fêmeas que serão usadas para a produção de leite, como também os bois que nascem na propriedade. Além da inserção de novo registro de animais que sejam comprados e trazidos para dentro da propriedade.

Para visualizar os dados de cada animal, o método “detalhes” carrega as listas de todas as classes que estão relacionadas ao mesmo, como a inseminação, reprodução, peso e alimentação, conforme apresenta a Listagem 9.

Listagem 9 – Lista os detalhes do animal

```

@GetMapping("/detalhes/{idAnimal}")
public ModelAndView detalhes(@PathVariable Long idAnimal, Inseminacao
inseminacao, Reproducao reproducao,
    PesoAnimal pesoAnimal, Lactacao lactacao,
CadastroExigencia cadastroExigencia, Exigencia exigencia, Animal animal,
Prontuario prontuario) {
    ModelAndView modelAndView = new
ModelAndView("animal/detalhes");

    //reproducao.getFlagVerdadeiroFalso();
    modelAndView.addObject(inseminacao);
    modelAndView.addObject(reproducao);
    modelAndView.addObject(pesoAnimal);
    modelAndView.addObject(lactacao);
    modelAndView.addObject(cadastroExigencia);
    modelAndView.addObject(exigencia);
    modelAndView.addObject(animal);
    modelAndView.addObject(prontuario);
    modelAndView.addObject("idadeAnimal",
calcularIdadeEmDias(animalRepository
.getOne(idAnimal)));
    modelAndView.addObject("diasLactacao",
calcularDiasEmLactacao(idAnimal));
    modelAndView.addObject("quantidadeLactacoes",
reproducaoRepository
.countByQuantidadeLactacoes(idAnimal));
    modelAndView.addObject("quantidadeInseminacoes",

```



```

inseminacaoRepository.countByQuantidadeInseminacoes("Confirmado",idAnimal));
    modelAndView.addObject("faseLactacao",
faseProducaoLeiteAnimal(idAnimal));
    modelAndView.addObject("animaisDetalhe",
animalRepository.getOne(idAnimal));
    modelAndView.addObject("inseminacaoList", inseminacaoRepository
.findByAnimalId(idAnimal));
    modelAndView.addObject("pesosList", pesoAnimalRepository
.findByAnimalId(idAnimal));
    modelAndView.addObject("escoreList",
EscoreCondicaoCorporal.values());
    modelAndView.addObject("touroList",
touroRepository.findByPropriedadeId(animalRepository.getOne(idAnimal)
.getPropriedade().getId()));
    modelAndView.addObject("reproducaoList", reproducaoRepository
.findByAnimailId(idAnimal));
    modelAndView.addObject("lactacaoList", lactacaoRepository
.findByAnimalId(idAnimal));
    modelAndView.addObject("exigenciasList", exigenciaRepository
.findByAnimalId(idAnimal));
    modelAndView.addObject("propriedades",propriedadeRepository
.getOne(animalRepository.getOne(idAnimal).getPropriedade().getId()));
    modelAndView.addObject("prontuarioList",
prontuarioRepository.findByAnimailId(idAnimal));
    modelAndView.addObject("filhosList",
animalRepository.findByFilhosAnimal(idAnimal));

    return modelAndView;
}

```

Fonte: Autoria própria.

Durante todo o processo de balanceamento, a fase da produção que o animal se encontra, é vital para que os cálculos dos nutrientes oferecidos sejam corretos para a exigência nutricional do animal. Essa atualização ocorre, quando se insere um registro de peso, parto, inseminação, lactação ou verificado quando uma nova interface é aberta.

O método “verificarAnimalSecaOuPreParto”, apresentado na Listagem 10, é executado quando os dados da propriedade são acessados e são listados os *cards* dos animais. Esse método exibe a lista de todos os animais da propriedade, e com uma estrutura de repetição faz a consulta se o animal está aguardando um parto. Se tiver um registro, verifica se está dentro do período de secagem, se sim, o animal está na fase de seca. Caso verifique que está no período de pré-parto, então esta será a fase. Se não estiver em nenhum desses períodos, não há alteração.

Listagem 10 – Método para verificar fase do animal

```

private void verificarAnimalSecaOuPreParto(Long idPropriedade) {
    List<Animal> animaisList = animalRepository
.findByPropriedadeId(idPropriedade);
    LocalDate hoje = LocalDate.now();

```

```

        if(animaisList != null) {
            for(Animal animais :animaisList) {
                Reproducao ultimaReproducao = reproducaoRepository
                .findByUltimaReproducao(animais.getId());
                if(ultimaReproducao != null) {
                    LocalDate dataInicioPreParto =
                    ultimaReproducao.getDataPrevistaParto().plusDays(-20);
                    if((ultimaReproducao.getDataSecagem().equals(hoje) ||
                    ultimaReproducao.getDataSecagem().isBefore(hoje)) &&
                    hoje.isBefore(dataInicioPreParto)) {
                        animais.setFaseAnimal(FaseAnimal.SECA);
                    }else if((dataInicioPreParto.equals(hoje) ||
                    dataInicioPreParto.isBefore(hoje) ) && ultimaReproducao.getDataParto() ==
                    null){
                        animais.setFaseAnimal(FaseAnimal.PREPARTO);
                    }
                }
                animalRepository.save(animais);
            }
        }
    }
}

```

Fonte: Autoria própria.

Também é realizado o cálculo de quantos dias o animal está em lactação e se está no início, meio ou fim do período de produção de leite, conforme apresenta a Listagem 11.

Listagem 11 – Cálculo dos dias em lactação

```

public long calcularDiasEmLactacao(Long idAnimal) {
    List<Reproducao> reproducaoList =
    reproducaoRepository.findTop1ByAnimalIdOrderByDataPartoDesc(idAnimal);
    LocalDate dataInicioLactacao = null;
    for (Reproducao r : reproducaoList) {
        dataInicioLactacao = r.getDataParto();
        System.out.println( dataInicioLactacao );
        break;
    }
    if(dataInicioLactacao == null) {
        return 0;
    }else {
        LocalDate hoje = LocalDate.now();
        long dias = dataInicioLactacao.until(hoje, ChronoUnit.DAYS);
        return dias;
    }
}

public String faseProducaoLeiteAnimal(Long idAnimal) {
    long diasLactacao = calcularDiasEmLactacao(idAnimal);
    if(diasLactacao == 0) {
        return "";
    }else if(diasLactacao < 101 && diasLactacao > 0) {
        return "Início";
    }else if(diasLactacao > 100 && diasLactacao < 201) {
        return "Meio";
    }
}

```

```

    }else {
        return "Final";
    }
}

```

Fonte: Autoria própria.

Para manter as inseminações, a classe *controller* tem o método para salvar uma nova inseminação, fazendo as verificações necessárias, primeiro se está apta para a reprodução e, em caso positivo, uma validação é realizada para que não haja duas inseminações ao mesmo tempo ou gerar um novo registro mesmo com uma gestação em andamento.

Decorridos 60 dias é possível verificar se o embrião se desenvolveu ou não. No primeiro caso, o método “*prenhezConfirmada*” é invocado para atualizar o *status* da inseminação para “confirmado”, gerando um novo registro de reprodução, com data prevista do parto e data de secagem. No segundo caso, o método “*prenhezNaoConfirmada*”, apenas altera o *status* para “não prenha”. A Listagem 12 apresenta a estrutura desses métodos.

Listagem 12 – Método para salvar uma nova inseminação

```

@PostMapping("/{idAnimal}")
public ResponseEntity<?> salvar(@Valid Inseminacao inseminacao,
@PathVariable long idAnimal, BindingResult result,
RedirectAttributes attributes) {
    Animal animal = animalRepository.getOne(idAnimal);
    //verifica se está apta a reprodução
    if(animal.getAptaReproducao()) {
        Inseminacao ultimaInseminacao = inseminacaoRepository
        .findTop1ByAnimalIdOrderByDataInseminacaoDesc(idAnimal);
        //verifica se o animal não tem nenhuma inseminação ou se as que tem, já teve
        o processo de confirmação da prenhez.
        if(ultimaInseminacao == null ||
!ultimaInseminacao.getStatusInseminacao().equals("Aguardando")) {
            try {
                //verifica se tem alguma reproducao aguardando parto, caso tenha não
                pode ser inserido nova inseminação
                if(reproducaoRepository.findByUltimaReproducao(idAnimal) != null) {
                    return new ResponseEntity<>("Animal está com gestação em andamento.
                    Aguardando parto!", HttpStatus.OK);
                }else {
                    inseminacao.setAnimal(animalRepository.getOne(idAnimal));
                    inseminacao.setStatusInseminacao("Aguardando");
                    inseminacaoRepository.save(inseminacao);
                    return new ResponseEntity<>("Registro salvo com sucesso!",
                    HttpStatus.OK);
                }
            } catch (Exception e) {
                return new ResponseEntity<>("Erro!!", HttpStatus.BAD_REQUEST);
            }
        }else if(ultimaInseminacao.getStatusInseminacao().equals("Aguardando")) {

```

```

        return new ResponseEntity<>("Ainda tem uma inseminação aguardando
confirmação!", HttpStatus.OK);
    }else {
        return new ResponseEntity<>("Erro!!", HttpStatus.BAD_REQUEST);
    }
    }else {
        return new ResponseEntity<>("Não está apta a reprodução!!",
HttpStatus.OK);
    }
}

@PostMapping("/prenhezConfirmada/{idInseminacao}")
public ResponseEntity<?> prenhezConfirmada(Inseminacao inseminacao,
@PathVariable long idInseminacao, BindingResult result, RedirectAttributes
attributes){
    if (result.hasErrors()) {
        return new ResponseEntity<>(result.getAllErrors(),
HttpStatus.BAD_REQUEST);
    }
    inseminacao = inseminacaoRepository.getOne(idInseminacao);
    List<Reproducao> ReproducaoAnimalList = reproducaoRepository
.findByReproducaoAnimal(inseminacao.getAnimal().getId(),
inseminacao.getId());
    //verifica se existe já uma reprodução para aquela inseminação
    for (Reproducao reproducao : ReproducaoAnimalList) {
        if(inseminacao.getId() == reproducao.getInseminacao().getId()) {

            return new ResponseEntity<>(result.getAllErrors(),
HttpStatus.BAD_REQUEST);

        }
    }

    Reproducao reproducao = new Reproducao();
    reproducao.setDataPrevistaParto(calcularDataPrevistaParto(inseminação
.getDataInseminacao()));
    reproducao.setDataSecagem(calcularDataSecagem(calcularDataPrevistaParto
(inseminacao.getDataInseminacao())));
    reproducao.setInseminacao(inseminacaoRepository.getOne(idInseminacao));
    reproducaoRepository.save(reproducao);
    inseminacao.setStatusInseminacao("Confirmado");
    inseminacaoRepository.save(inseminacao);
    attributes.addFlashAttribute("mensagem", "Registro salvo com
sucesso!");
    return new ResponseEntity<>(HttpStatus.OK);
}

@PostMapping("/prenhezNaoConfirmada/{idInseminacao}")
public ResponseEntity<?> prenhezNaoConfirmada(Inseminacao inseminacao,
@PathVariable long idInseminacao, BindingResult result,
RedirectAttributes attributes){
    if (result.hasErrors()) {
        return new ResponseEntity<>(result.getAllErrors(),
HttpStatus.BAD_REQUEST);
    }
    Reproducao reproducao = (reproducaoRepository
.findByReproducaoInseminacao(idInseminacao));
    if( reproducao != null) {
        reproducaoRepository.delete(reproducao);
        inseminacao = inseminacaoRepository.getOne(idInseminacao);
    }
}

```

```

        inseminacao.setStatusInseminacao("Não Prenha");
        inseminacaoRepository.save(inseminacao);
    }else {
        inseminacao = inseminacaoRepository.getOne(idInseminacao);
        inseminacao.setStatusInseminacao("Não Prenha");
        inseminacaoRepository.save(inseminacao);
    }
    attributes.addFlashAttribute("mensagem", "Registro salvo com
sucesso!");
    return new ResponseEntity<>(HttpStatus.OK);
}

```

Fonte: Autoria própria.

Quando o parto ocorre de fato, o método “salvar” do *controller* de reprodução faz a inserção de um novo registro, se o animal tiver nascido vivo e for uma fêmea, conforme apresenta a Listagem 13.

Listagem 13 – Método para salvar o nascimento de uma fêmea

```

@PostMapping("/{idReproducao}")
public ModelAndView salvar(@Valid Reproducao reproducao, @PathVariable Long
idReproducao, BindingResult result,
RedirectAttributes attributes) {
    reproducao = reproducaoRepository.getOne(idReproducao);
    reproducao.setDataParto(reproducao.getDataParto());
    reproducaoRepository.save(reproducao);
    Animal animal = new Animal();
    animal.setDataNascimento(reproducao.getDataParto());
    animal.setMae(reproducao.getInseminacao().getAnimal());
    animal.setTouro(reproducao.getInseminacao().getTouro());
    animal.setFaseAnimal(FaseAnimal.LACTACAO);
    ModelAndView modelAndView = new
ModelAndView("animal/cadastroNascimento");
    modelAndView.addObject(animal);
    return modelAndView;
}

```

Fonte: Autoria própria.

Se o bezerro nascido for macho, o *controller* de bois nascidos na propriedade vai gerar o registro e atualizar os dados da reprodução, conforme exibido na Listagem 14.

Listagem 14 – Método para salvar o nascimento de um boi

```

@PostMapping("/salvar/{idReproducao}")
public ModelAndView salvar(@Valid BoisNascidosPropriedade
boisNascidosPropriedade, @PathVariable Long idReproducao, BindingResult
result,
RedirectAttributes attributes) {
    Reproducao reproducao = reproducaoRepository.getOne(idReproducao);
    reproducao.setDataParto(boisNascidosPropriedade.getDataNascimento());
    reproducao.setVivo(true);
    Propriedade propriedade = propriedadeRepository

```

```

        .getOne(reproducao.getInseminacao().getAnimal().getPropriedade().getId());
        boisNascidosPropriedade.setStatus(true);
        boisNascidosPropriedade.setMae(reproducao.getInseminacao()
        .getAnimal());
        boisNascidosPropriedade.setPai(reproducao.getInseminacao()
        .getTouro());
        boisNascidosPropriedade.setPropriedade(propriedade);
        boisNascidosPropriedadeRepository.save(boisNascidosPropriedade);
        Animal mae = animalRepository.findByReproducaoId(idReproducao);
        mae.setFaseAnimal(FaseAnimal.LACTACAO);
        animalRepository.save(mae);
        Long idMae = mae.getId();
        attributes.addFlashAttribute("mensagem", "Registro salvo com
        sucesso!");
        return new ModelAndView("redirect:/animaisController/detalhes/" +
        idMae);
    }

```

Fonte: Autoria própria.

E uma terceira situação é quando ocorre o aborto ou o bezerro nasce morto, conhecido como natimorto, nesse caso o método “salvarProntuario”, apresentado na Listagem 15, do *controller* de prontuário irá gerar um registro e atualizar a reprodução referenciada.

Listagem 15 – Método para salvar prontuário

```

@PostMapping("/salvar/{idReproducao}")
public ModelAndView salvarProntuario( @Valid Prontuario prontuario,
@PathVariable Long idReproducao, BindingResult result, RedirectAttributes
attributes) {
    Animal mae = animalRepository.findByReproducaoId(idReproducao);
    Long idMae = mae.getId();
    Reproducao reproducao = reproducaoRepository.getOne(idReproducao);
    reproducao.setDataParto(prontuario.getDataOcorrencia());
    reproducao.setVivo(false);
    prontuario.setAnimal(mae);
    mae.setFaseAnimal(FaseAnimal.LACTACAO);
    prontuarioRepository.save(prontuario);
    attributes.addFlashAttribute("mensagem", "Registro salvo com
    sucesso!");
    return new ModelAndView("redirect:/animaisController/detalhes/" +
    idMae);
}

```

Fonte: Autoria própria.

O *controller* de pesos dos animais possui métodos que irão salvar e avaliar se o peso está adequado para a idade e porte da raça, comparando o peso atual com a tabela de pesos esperados.

A Listagem 16 apresenta o cálculo de ganho médio diário, ou seja, pega o valor do peso que foi inserido no *input*, menos o peso anterior e divide pela quantidade de dias entre a data atual e data do último registro, retornando o valor.

Listagem 16 – Calcular o GMD

```
private Double calcularGanhoMedioDiario(CadastroPeso cadastroPeso) {
    Double gmdReal = 0.0;
    Double pesoAnterior = 0.0;;
    if(pesoAnimalRepository.findByAnimalId(cadastroPeso.getIdAnimal())
.isEmpty()) {
        return gmdReal;
    }else {
        PesoAnimal pesoAnimal =
pesoAnimalRepository.findTop1ByAnimalIdOrderByDataPesagemDesc(cadastroPeso
.getIdAnimal());
        pesoAnterior = pesoAnimal.getPeso();
        gmdReal = (cadastroPeso.getPeso() - pesoAnterior) /
calcularDiasEntreDuasDatas(cadastroPeso);
        return gmdReal;
    }
}
```

Fonte: Autoria própria.

O método da Listagem 17, verifica junto a tabela pesos esperados, se o ganho médio diário está adequado para o animal, retornando a mensagem conforme a situação do peso. Se estiver alto, significa que o animal ganhou muito peso e se estiver baixo, indica que não atingiu o peso necessário para a idade. É importante que o retorno seja adequado, para o correto desenvolvimento do animal.

Listagem 17 – Calcular o status do GMD

```
private String calculaStatusGMD(Animal animal, CadastroPeso cadastroPeso) {
    Double GMD = calcularGanhoMedioDiario(cadastroPeso);
    PesoEsperado pesoEsperado = buscarPesoEsperado(animal, cadastroPeso);
    if(GMD > pesoEsperado.getGmdMax()) {
        return "Alto";
    }else if(GMD < pesoEsperado.getGmdMin()) {
        return "Baixo";
    }else {
        return "Bom";
    }
}
```

Fonte: Autoria própria.

O método apresentado na Listagem 18, verifica se o animal tem condições de começar uma gestação, avaliando o porte da raça, o peso e a idade, retornando verdadeiro caso atende os parâmetros.

Listagem 18 – Método para verificar se está apta para reprodução

```
private Boolean verificarAnimalAptaReproducao(Animal animal, CadastroPeso
cadastroPeso) {
    long idadeDias = calcularIdadeEmDias(animal, cadastroPeso);
    Boolean aptaReproducao = false;
    if(animal.getRaca().getPorteRaca().toString().equals("PEQUENA")) {
        if(cadastroPeso.getPeso() >= 250 && idadeDias > 365) {
            aptaReproducao = true;
        }
    } else if(animal.getRaca().getPorteRaca().toString().equals("MEDIA")) {
        if(cadastroPeso.getPeso() >= 300 && idadeDias > 396) {
            aptaReproducao = true;
        }
    } else if(animal.getRaca().getPorteRaca().toString().equals("GRANDE"))
    {
        if(cadastroPeso.getPeso() >= 350 && idadeDias > 396) {
            aptaReproducao = true;
        }
    }
    return aptaReproducao;
}
```

Fonte: Autoria própria.

A Listagem 19 apresenta outro método que faz a atualização da fase do animal no sistema. A fase de bezerra vai até os 179 dias e é alterada após atingir essa idade para a fase novilha, que se estende até ocorrer o primeiro parto. Esse método de atualização da fase foi inserido no *controller* de pesos, pois bezerra e novilha só ocorrem uma vez na vida do animal e como o peso é lançado, geralmente, apenas uma vez por mês, foi considerado que fazer essa validação nesse *controller*, evitaria processamento desnecessário.

Listagem 19 – Método para verificar a fase do animal

```
private String verificarAnimalFaseBezerraOuNovilha(Animal animal,
CadastroPeso cadastroPeso) {
    long idadeDias = calcularIdadeEmDias(animal, cadastroPeso);
    List<Reproducao> reproducao =
reproducaoRepository.findByAnimalId(cadastroPeso.getIdAnimal());
    if(idadeDias <= 180) {
        return "BEZERRA";
    } else if(idadeDias > 180 && reproducao.size() <= 1){//verifica se o
animal tem registro de reproducao
        int fase = 0;
        for(Reproducao rep : reproducao) {
            if(rep.getDataParto() == null) {AINDA//VERIFICA SE NÃO TEVE PARTO
                fase = 1;
            }
        }
        if(fase == 1) {
```



```

        return "NOVILHA";
    }else {
        return null;
    }
    }else {
        return null;
    }
}

```

Fonte: Autoria própria.

O *controller* de pesos tem dois métodos para salvar, um para registros individuais, e o apresentado da Listagem 20, que salva uma lista de pesos dos animais, recebidos por meio de requisição AJAX.

Listagem 20 – Método para salvar lista de pesos

```

@PostMapping("/salvarPesoRebanho")
public ResponseEntity<?> salvarRebanho(@RequestBody @Valid
List<CadastroPeso> cadastroPesoList, BindingResult result, Model
model, RedirectAttributes attributes) {
    if (result.hasErrors()) {
        return new ResponseEntity<>(result.getAllErrors(),
HttpStatus.BAD_REQUEST);
    }
    for(CadastroPeso cadastroPeso : cadastroPesoList) {

        Animal animal =
animalRepository.getOne(cadastroPeso.getIdAnimal());
        animal.setAptaReproducao(verificarAnimalAptaReproducao(animal,
cadastroPeso));
        if(verificarAnimalFaseBezerraOuNovilha(animal, cadastroPeso)
== "BEZERRA") {
            animal.setFaseAnimal(FaseAnimal.BEZERRA);
            animalRepository.save(animal);
        }else if(verificarAnimalFaseBezerraOuNovilha(animal,
cadastroPeso) == "NOVILHA") {
            animal.setFaseAnimal(FaseAnimal.NOVILHA);
            animalRepository.save(animal);
        }

        PesoAnimal pesoAnimal = new PesoAnimal();
        pesoAnimal.setDataPesagem(cadastroPeso.getDataPesagem());
        pesoAnimal.setPeso(cadastroPeso.getPeso());
        pesoAnimal.setEcc(cadastroPeso.getEcc());
        pesoAnimal.setAnimal(animal);
        pesoAnimal.setIdadeAtual( calcularIdadeEmDias(animal, cadastroPeso));
        pesoAnimal.setGanhoMedioDiarioGMD
(calcularGanhoMedioDiario(cadastroPeso));
        pesoAnimal.setStatusGMD(calculaStatusGMD(animal, cadastroPeso));
        pesoAnimal.setPesoEsperado(buscarPesoEsperado
(animal, cadastroPeso));
        pesoAnimalRepository.save(pesoAnimal);
    }

    attributes.addFlashAttribute("mensagem", "Registro salvo com
sucesso!");
    return new ResponseEntity<>(HttpStatus.OK);
}

```

}

Fonte: Autoria própria.

No *controller* das exigências estão todos os cálculos necessários para conhecer a necessidade de nutrientes do animal e salvar o resultado no banco de dados.

Todos os cálculos que serão apresentados foram reproduzidos das planilhas utilizadas pela equipe de pesquisa e extensão do IDR-Paraná. Fórmulas essas que tem a origem na literatura, mais especificamente no livro “nutrient requirements of dairy cattle do National Research Council (NRC)”.

Dessa forma, o que se buscou nos métodos foi trazer os dados dos animais corretos, conforme estavam apresentados nas planilhas, não entrando no mérito de questionar o motivo de determinadas operações aritméticas.

São dez tipos de exigências diferentes calculadas, que utilizam, basicamente, dados do próprio animal, como peso, idade, fase da vida que o animal se encontra, produção de leite.

A primeira é o cálculo da exigência de Materia Seca (Listagem 21), na qual é verificado em que fase o animal está e a temperatura do ambiente, inserida pelo usuário, diferenciando a formula conforme esses critérios. Ainda quando o animal está na fase de lactação, é verificado há quantos dias está produzindo leite, para que possa ser realizado o cálculo correto.

Listagem 21 – Exigência de matéria seca

```
private Double exigenciaMateriaSecaKgDia (CadastroExigencia
cadastroExigencia) {
    Animal animal = animalRepository.getOne(cadastroExigencia.getIdAnimal());
    try {
        long quantidadeLactacoes =
reproducaoRepository.countByQuantidadeLactacoes(cadastroExigencia
.getIdAnimal());
        long diasGestacao = calcularDiasEmGestacao(cadastroExigencia);

        Double proximoPeso =
calculaPesoEstimadoVacaProximoBalanceamento(cadastroExigencia);
        Double calculoLogaritmo = null;
        Double ingestaoMateriaSecaKgDia = null;

        if((animal.getFaseAnimal().toString().equals("PREPARTO") ||
animal.getFaseAnimal().toString().equals("SECA")) &&
cadastroExigencia.getTemperatura() >= 20 && quantidadeLactacoes >1){

            calculoLogaritmo = Math.log((0.16 * diasGestacao) - 282);
            ingestaoMateriaSecaKgDia = (1.97 - 0.75 *
(calculoLogaritmo / 100) * proximoPeso) * (1 -
```

```

(cadastroExigencia.getTemperatura() - 20)
* 0.005922);          return ingestaoMateriaSecaKgDia;

        }else if((animal.getFaseAnimal().toString().equals("PREPARTO")
|| animal.getFaseAnimal().toString().equals("SECA")) &&
cadastroExigencia.getTemperatura() < 20 && quantidadeLactacoes >1) {
        calculoLogaritmo = Math.Log((0.16 * diasGestacao) - 282);
        ingestaoMateriaSecaKgDia = (1.97 - 0.75 *
(calculoLogaritmo / 100) * proximoPeso);
        return ingestaoMateriaSecaKgDia;
        }else if(animal.getFaseAnimal().toString().equals("LACTACAO")
&& cadastroExigencia.getTemperatura() >= 20) {
        long diasLactacao =
calcularDiasEmLactacao(cadastroExigencia);
        Double leiteCorrigido =
calculaLeiteCorrigido4PorcentoDeGordura(cadastroExigencia);
        if(diasLactacao <= 30) {
        calculoLogaritmo = Math.Log((-0.192 * 60 / 7 + 3.67));
        ingestaoMateriaSecaKgDia = (0.372 * leiteCorrigido +
0.0968 * Math.pow(proximoPeso, 0.75)) * ((1 - calculoLogaritmo) * (1 -
(cadastroExigencia.getTemperatura() - 20) * 0.005922));

        return ingestaoMateriaSecaKgDia;
        }else {
        calculoLogaritmo = Math.Log((-0.192 * diasLactacao / 7 +
3.67));
        ingestaoMateriaSecaKgDia = (0.372 * leiteCorrigido +
0.0968 * Math.pow(proximoPeso, 0.75)) * ((1 - calculoLogaritmo) * (1 -
(cadastroExigencia.getTemperatura() - 20) * 0.005922));
        return ingestaoMateriaSecaKgDia;
        }
        }else if(animal.getFaseAnimal().toString().equals("LACTACAO")
&& cadastroExigencia.getTemperatura() < 20){
        long diasLactacao = calcularDiasEmLactacao(cadastroExigencia);
        Double leiteCorrigido =
calculaLeiteCorrigido4PorcentoDeGordura(cadastroExigencia);
        if(diasLactacao <= 30) {
        calculoLogaritmo = Math.Log((-0.192 * 60 / 7 + 3.67));
        ingestaoMateriaSecaKgDia = (0.372 * leiteCorrigido + 0.0968 *
Math.pow(proximoPeso, 0.75)) * ((1 - calculoLogaritmo));

        return ingestaoMateriaSecaKgDia;
        }else {
        calculoLogaritmo = Math.Log((-0.192 * diasLactacao / 7 +
3.67));
        ingestaoMateriaSecaKgDia = (0.372 * leiteCorrigido + 0.0968 *
Math.pow(proximoPeso, 0.75)) * ((1 - calculoLogaritmo));

        return ingestaoMateriaSecaKgDia;
        }
        }else {
        return 0.0;
        }
    } catch (Exception e) {
        return 0.0;
    }
}

```

Fonte: Autoria própria.

A Listagem 22, apresenta alguns métodos da exigência de energia líquida. Para se chegar a um resultado, é calculado outros tipos de energias líquidas. A energia líquida de manutenção, que seria a quantidade de energia para a subsistência do animal, a energia líquida de lactação, que precisa projetar a quantidade de leite para o próximo balanceamento, utiliza o teor de proteína e gordura na composição do leite para gerar o resultado, isso se o animal estiver na fase de lactação. Já para calcular a energia líquida, avalia que o animal precisa ganhar ou perder peso até o próximo balanceamento, então o método “energiaLiquidaGanhoOuPerdaMcalDia” vai fazer essa verificação. Se considera o peso atual e o peso estimado para o próximo balanceamento, retornando o valor da energia líquida correspondente a perder ou ganhar de peso.

O retorno de todas as energias líquidas é somado, gerando o valor da energia líquida em Mcal/Dia, e o valor final utilizado no balanceamento é o Mcal/Kg.

Listagem 22 – Exigência de energia líquida

```
private Double exigenciaEnergiaLiquidaMcalKg(CadastroExigencia
cadastroExigencia) {
    Double exigenciaEnergiaLiquidaMcalKg = null;
    Double exigenciaEnergiaLiquidaMcalDia =
exigenciaEnergiaLiquidaMcalDia(cadastroExigencia);
    Double exigenciaMateriaSecaKgDia =
exigenciaMateriaSecaKgDia(cadastroExigencia);
    if(exigenciaMateriaSecaKgDia == 0.0) {
        exigenciaEnergiaLiquidaMcalKg = 0.0;
        return exigenciaEnergiaLiquidaMcalKg;
    }else {
        exigenciaEnergiaLiquidaMcalKg = exigenciaEnergiaLiquidaMcalDia
/ exigenciaMateriaSecaKgDia;
        return exigenciaEnergiaLiquidaMcalKg;
    }
}

private Double exigenciaEnergiaLiquidaMcalDia(CadastroExigencia
cadastroExigencia) {
    Double exigenciaEnergiaLiquidaMcalDia = null;
    exigenciaEnergiaLiquidaMcalDia =
energiaLiquidaManutencaoMcalDia(cadastroExigencia) +
energiaLiquidaLactacaoMcalDia(cadastroExigencia) +
energiaLiquidaGestacaoMcalDia(cadastroExigencia) +
energiaLiquidaGanhoOuPerdaMcalDia(cadastroExigencia) ;
    return exigenciaEnergiaLiquidaMcalDia;
}
```

Fonte: Autoria própria.

A Listagem 23 apresenta o método para o cálculo da exigência em nutrientes digestíveis totais.

Listagem 23 – Exigência de nutrientes digestíveis totais

```
private Double exigenciaEmNutrientesDigestiveisTotaisNDT(CadastroExigencia cadastroExigencia){
    Double exigenciaEmNutrientesDigestiveisTotaisNDT =
    (exigenciaEnergiaLiquidaMcalDia(cadastroExigencia) / 0.0245 + 0.12) / 100;
    return exigenciaEmNutrientesDigestiveisTotaisNDT;
}
```

Fonte: Autoria própria.

A Listagem 24 apresenta o cálculo da exigência de energia metabolizável.

Listagem 24 – Exigência de energia metabolizável

```
private Double exigenciaEnergiaMetabolizavel(CadastroExigencia cadastroExigencia) {
    Double exigenciaNDTporcentagem =
    exigenciaEmNutrientesDigestiveisTotaisPorcentagem(cadastroExigencia);
    Double exigenciaEnergiaMetabolizavel = 0.0;
    exigenciaEnergiaMetabolizavel = (1.01 * (0.04409 *
    exigenciaNDTporcentagem) - 0.45);
    return exigenciaEnergiaMetabolizavel;
}
```

Fonte: Autoria própria.

O cálculo da exigência de proteína degradável no rúmen é feito pelo método da Listagem 25. Nele é utilizado o peso estimado para o próximo balanceamento, os valores do teor de proteína e de gordura do último registro de lactação, além da exigência de matéria seca.

Listagem 25 – Exigência de proteína degradável no rúmen

```
private Double exigenciaProteinaDegradavelRumenEmKg(CadastroExigencia cadastroExigencia) {
    try {
        Animal animal = animalRepository.findOne(cadastroExigencia.getIdAnimal());
        Double proximoPesoVaca =
        calculaPesoEstimadoVacaProximoBalanceamento(cadastroExigencia);
        Lactacao lactacao =
        lactacaoRepository.findTop1ByAnimalIdOrderByDataMedicaoDesc(cadastroExigencia.getIdAnimal());
        Double exigenciaMateriaSecaKgDia =
        exigenciaMateriaSecaKgDia(cadastroExigencia);
        Double proteinaDegradavelRumen = null;

        if(animal.getFaseAnimal().toString().equals("SECA") ||
        animal.getFaseAnimal().toString().equals("PREPARTO")) {
            proteinaDegradavelRumen = ((0.241 - (0.98 * (-1.67 -
            (0.00513 * proximoPesoVaca) - (0.597 * lactacao.getTeorGordura())) + (2.53 *
            lactacao.getTeorProteina())) + (0.977 * ((8.83 - (0.00483 * proximoPesoVaca)
```

```

+ (0.119 * lactacao.getTeorProteina()) - (0.706 * lactacao.getTeorGordura())
+ (2.54 * lactacao.getTeorProteina())) / 100) * exigenciaMateriaSecaKgDia));
    return proteinaDegradavelRumen;
} else if (animal.getFaseAnimal().toString().equals("LACTCAO")) {
    Double producaoLeiteProjetadaPico =
calculaProducaoLeiteKgDiaProjetadaParaPico(cadastroExigencia);
    proteinaDegradavelRumen = ((0.241 - (0.98 * (-1.67 -
(0.00513 * proximoPesoVaca) + (0.13 * producaoLeiteProjetadaPico) - (0.597 *
lactacao.getTeorGordura()) + (2.53 * lactacao.getTeorProteina())))) + (0.977 *
((8.83 - (0.00483 * proximoPesoVaca) + (0.119 * lactacao.getTeorProteina()) -
(0.706 * lactacao.getTeorGordura()) + (2.54 * lactacao.getTeorProteina())))) /
100) * exigenciaMateriaSecaKgDia));
    return proteinaDegradavelRumen;

} else {
    return proteinaDegradavelRumen = 0.0;
}
} catch (Exception e) {
    return 0.0;
}
}

```

Fonte: Autoria própria.

O método apresentado na Listagem 26, calcula a exigência de proteína não degradável no Rúmen. Apresenta a mesma estrutura da Listagem 25, diferenciando apenas nas operações aritméticas.

Listagem 26 – Exigência de proteína não degradável no rúmen

```

private Double exigenciaProteinaNAODegradavelRumenEmKg(CadastroExigencia
cadastroExigencia) {
    Animal animal =
animalRepository.getOne(cadastroExigencia.getIdAnimal());
    try {
        Double proximoPesoVaca =
calculaPesoEstimadoVacaProximoBalanceamento(cadastroExigencia);
        Lactacao lactacao =
lactacaoRepository.findTop1ByAnimalIdOrderByDataMedicaoDesc(cadastroExigencia
.getIdAnimal());
        Double exigenciaMateriaSecaKgDia =
exigenciaMateriaSecaKgDia(cadastroExigencia);
        Double proteinaNAODegradavelRumen = null;
        if (animal.getFaseAnimal().toString().equals("SECA") ||
animal.getFaseAnimal().toString().equals("PREPARTO")) {
            proteinaNAODegradavelRumen = (((-1.67 - (0.00513 *
proximoPesoVaca) - (0.597 * lactacao.getTeorGordura()) + (2.53 *
lactacao.getTeorProteina()) / 100) * exigenciaMateriaSecaKgDia));
            return proteinaNAODegradavelRumen;
        } else if (animal.getFaseAnimal().toString().equals("LACTCAO")) {
            Double producaoLeiteProjetadaPico =
calculaProducaoLeiteKgDiaProjetadaParaPico(cadastroExigencia);
            proteinaNAODegradavelRumen = (((-1.67 - (0.00513 *
proximoPesoVaca) + (0.13 * producaoLeiteProjetadaPico) - (0.597 *
lactacao.getTeorGordura()) + (2.53 * lactacao.getTeorProteina())) / 100) *
exigenciaMateriaSecaKgDia);
            return proteinaNAODegradavelRumen;
        }
    }
}

```

```

        }else {
            return proteinaNAODegradavelRumen = 0.0;
        }
    } catch (Exception e) {
        return 0.0;
    }
}

```

Fonte: Autoria própria.

A exigência de proteína metabolizável é calculada conforme o método apresentado na Listagem 27. Para retornar o valor, precisa realizar a soma dos valores de métodos específicos para cada proteína, sendo elas: proteína metabolizável da descamação, proteína metabolizável da urina, proteína metabolizável fecal, proteína metabolizável endógena, proteína metabolizável da lactação e proteína metabolizável na gestação.

Listagem 27 – Exigência de proteína metabolizável

```

private Double exigenciaProteinaMetabolizavelKgDia(CadastroExigencia
cadastroExigencia) {
    try {
        Double proteinaMetabolizavelTotal = null;
        Double exigenciaProteinaBruta = null;
        proteinaMetabolizavelTotal =
proteinaMetabolizavelDescamacaoGdia(cadastroExigencia) +
proteinaMetabolizavelUrinaGdia(cadastroExigencia) +
proteinaMetabolizavelFecalGdia(cadastroExigencia) +
proteinaMetabolizavelEndogenaGdia(cadastroExigencia) +
proteinaMetabolizavelLactacao(cadastroExigencia) +
proteinaMetabolizavelGestacao(cadastroExigencia);
        exigenciaProteinaBruta = ((proteinaMetabolizavelTotal /
0.67)/1000);
        return exigenciaProteinaBruta;
    } catch (Exception e) {
        return 0.0;
    }
}

```

Fonte: Autoria própria.

A próxima exigência é a de proteína bruta, conforme apresenta a Listagem 28, que utiliza o retorno da proteína metabolizável, para transformar em bruta.

Listagem 28 – Exigência de proteína bruta

```

private Double exigenciaProteinaBrutaKg(CadastroExigencia cadastroExigencia)
{
    try {
        Double proteinaBruta = null;
        Double proteinaMetabolizavel =
exigenciaProteinaMetabolizavelKgDia(cadastroExigencia);
    }
}

```

```

        proteinaBruta = proteinaMetabolizavel * 0.93;
        return proteinaBruta;
    } catch (Exception e) {
        return 0.0;
    }
}

```

Fonte: Autoria própria.

A Listagem 29 apresenta o método que realiza o cálculo da exigência de cálcio. Sendo necessária a utilização de outros métodos para o cálcio que se perde em situações específicas, sendo eles: o cálcio fecal, cálcio da urina, cálcio na gestação, cálcio na lactação.

Listagem 29 – Exigência de cálcio

```

private Double exigenciaCalcioKgDia(CadastroExigencia cadastroExigencia) {
    try {
        Double exigenciaCalcioGdia =
            ((calcioFecalGdia(cadastroExigencia) + calcioUrinaGdia(cadastroExigencia) +
            calcioGestacaoGdia(cadastroExigencia) +
            calcioLactacaoGdia(cadastroExigencia))/0.6);
        Double exigenciaCalcioKgDia = exigenciaCalcioGdia / 1000;
        return exigenciaCalcioKgDia;
    } catch (Exception e) {
        return 0.0;
    }
}

```

Fonte: Autoria própria.

Para a exigência de fósforo, a Listagem 30 apresenta a estrutura do método, usando os cálculos de fósforo fecal, fósforo da urina, fósforo na gestação e fósforo na lactação.

Listagem 30 – Exigência de fósforo

```

private Double exigenciaFosforokgDia(CadastroExigencia cadastroExigencia) {
    try {
        Double exigenciaFosforoGdia =
            ((fosforoFecalGdia(cadastroExigencia) + fosforoUrinaGdia(cadastroExigencia)
            + fosforoGestacaoGdia(cadastroExigencia) +
            fosforoLactacaoGdia(cadastroExigencia)) / 0.7);
        Double exigenciaFosforoKgDia = exigenciaFosforoGdia/1000;
        return exigenciaFosforoKgDia;
    } catch (Exception e) {
        return 0.0;
    }
}

```

Fonte: Autoria própria.

Durante todos os cálculos de exigência são usados dados específicos dos animais, trazidos do banco através de consultas.

O método da Listagem 31, busca no banco o último registro de peso no animal e com o valor de perda ou ganho de peso, que o usuário digita, se faz a estimativa do peso no animal para o próximo balanceamento, que é feito a cada 30 dias.

Listagem 31 – Calculo do peso estimado para o próximo balanceamento

```
private Double calculaPesoEstimadoVacaProximoBalanceamento
(CadastroExigencia
cadastroExigencia) {
    Double proximoPeso = null;
    PesoAnimal pesoVivo =
pesoAnimalRepository.findTop1ByAnimalIdOrderByDataPesagemDesc
(cadastroExigencia.getIdAnimal());
    if(pesoVivo == null) {
        return 0.0;
    }else {
        if(cadastroExigencia.getPerdaPeso() > 0 && pesoVivo != null) {
            proximoPeso = (pesoVivo.getPeso() -
(cadastroExigencia.getPerdaPeso() * (365/12)));
            return proximoPeso;
        }else if(cadastroExigencia.getGanhoPeso() > 0 && pesoVivo !=
null) {
            proximoPeso = (pesoVivo.getPeso() +
(cadastroExigencia.getPerdaPeso() * (365/12)));
            return proximoPeso;
        }else {
            proximoPeso = pesoVivo.getPeso();
            return proximoPeso;
        }
    }
}
```

Fonte: Autoria própria.

O método da Listagem 32, consulta a última data de inseminação do animal, para calcular quantos dias têm sua gestação.

Listagem 32 – Cálculo dos dias em gestação

```
private long calcularDiasEmGestacao(CadastroExigencia cadastroExigencia) {
    Inseminacao inseminacao = inseminacaoRepository
.findTop1ByAnimalIdOrderByDataInseminacaoDesc(cadastroExigencia
.getIdAnimal());
    if(inseminacao == null) {
        return 0;
    }else {
        LocalDate dataInseminacao = inseminacao.getDataInseminacao();
        LocalDate hoje = LocalDate.now();
        long dias = dataInseminacao.until(hoje, ChronoUnit.DAYS);
        return dias;
    }
}
```

```

    }
}

```

Fonte: Autoria própria.

Para calcular quantos dias o animal está em lactação, o método apresentado na Listagem 33, recupera do banco de dados a última data de parto.

Listagem 33 – Cálculo dos dias em lactação

```

private long calcularDiasEmLactacao(CadastroExigencia cadastroExigencia) {
    try {
        List<Reproducao> reproducaoList =
reproducaoRepository.findTop1ByAnimalIdOrderByDataPartoDesc(cadastroExigencia
.getIdAnimal());
        LocalDate dataIniciolactacao = null;
        for (Reproducao r : reproducaoList) {
            dataIniciolactacao = r.getDataParto();
            break;
        }
        long dias = 0;
        if(dataIniciolactacao == null) {
            return 0;
        }else {
            LocalDate hoje = LocalDate.now();
            dias = dataIniciolactacao.until(hoje, ChronoUnit.DAYS);
            return dias;
        }
    } catch (Exception e) {
        return 0;
    }
}

```

Fonte: Autoria própria.

A Listagem 34, corrige a gordura do leite, que vai ser utilizado nos cálculos de exigência. Para isso é recuperado do banco de dados o registro mais recente de lactação que foi inserido.

Listagem 34 – Cálculo do leite corrigido

```

private Double calculaLeiteCorrigido4PorcentoDeGordura(CadastroExigencia
cadastroExigencia) {
    Animal animal = animalRepository.getOne(cadastroExigencia
.getIdAnimal());
    try {
        Lactacao lactacao = lactacaoRepository
.findTop1ByAnimalIdOrderByDataMedicaoDesc(cadastroExigencia
.getIdAnimal());
        Double producaoLeiteDia = lactacao.getProducaoLeiteDia();
        if(animal.getFaseAnimal().toString().equals("LACTACAO") &&
producaoLeiteDia > 0 ) {
            Double teorGordura = lactacao.getTeorGordura();
            Double leiteCorrigido = null;
            leiteCorrigido = (0.4 * producaoLeiteDia) + (15 *

```

```

        (teorGordura / 100)) * producaoLeiteDia;
        return leiteCorrigido;
    }else {
        return 0.0;
    }
} catch (NullPointerException ex) {
    return 0.0;
}
}

```

Fonte: Autoria própria.

O método apresentado na Listagem 35, utilizado o cálculo de dias em gestação e o peso estimado da vaca para o próximo balanceamento, para calcular o peso esperado do bezerro que vai nascer.

Listagem 35 – Cálculo do peso estimado ao nascimento

```

private Double calculaPesoEstimadoBezerroAoNascimentoKg(CadastroExigencia
cadastroExigencia) {
    try {
        long diasGestacao = calcularDiasEmGestacao(cadastroExigencia);
        Double proximoPesoVaca =
calculaPesoEstimadoVacaProximoBalanceamento(cadastroExigencia);
        Double pesoEstimadoBezerro = null;
        if(diasGestacao < 190) {
            return 0.0;
        }else {
            pesoEstimadoBezerro = proximoPesoVaca*0.06275;
            return pesoEstimadoBezerro;
        }
    } catch (Exception e) {
        return 0.0;
    }
}

```

Fonte: Autoria própria.

O método apresentado na Listagem 36, tem a finalidade de projetar a produção de leite quando o animal atingir o pico, ou seja, o período de maior produção. Para isso é recuperado do banco de dados o último registro de lactação inserido e utiliza o cálculo dos dias em lactação para retorna o valor.

Listagem 36 – Cálculo da produção de leite projetada

```

private Double calculaProducaoLeiteKgDiaProjetadaParaPico(CadastroExigencia
cadastroExigencia) {
    Animal animal =
animalRepository.getOne(cadastroExigencia.getIdAnimal());
    try {
        Lactacao lactacao =
lactacaoRepository.findTop1ByAnimalIdOrderByDataMedicaoDesc(cadastroExigencia
.getIdAnimal());
        long diasLactacao = calcularDiasEmLactacao(cadastroExigencia);
    }
}

```

```

        Double producaoLeiteDia = lactacao.getProducaoLeiteDia();
        if (animal.getFaseAnimal().toString().equals("LACTACAO") &&
            diasLactacao >= 19 && diasLactacao <= 41) {
            producaoLeiteProjetada = producaoLeiteDia + 3;
            return producaoLeiteProjetada;
        } else {
            return producaoLeiteDia;
        }
    } catch (NullPointerException ex) {
        return 0.0;
    }
}

```

Fonte: Autoria própria.

No *controller* das formulações, o usuário pode escolher os alimentos, trazidos por meio da lista do método novo apresentado na Listagem 37.

Listagem 37 – Lista alimentos para escolha

```

@GetMapping("/novo")
public ModelAndView novo(FormulacaoAlimento formulacaoAlimento) {
    ModelAndView modelAndView = new
    ModelAndView("balanceamento/formulacao");
    modelAndView.addObject(formulacaoAlimento);
    modelAndView.addObject("alimentosList", alimentoRepository.findAll());
    return modelAndView;
}

```

Fonte: Autoria própria.

Após a escolha dos alimentos, o usuário salva a lista. O método salvar apresentado na Listagem 38, recebe a lista de alimentos escolhidos por meio de requisição AJAX e busca a lista das últimas exigências calculadas de cada animal. São duas estruturas de repetição, sendo que, na primeira é gerada uma nova formulação setando o *id* do animal e data que está sendo realizada e, na segunda, é criada a formulação/alimento, que contém os alimentos escolhidos, valor e é setado o *id* da formulação correspondente.

Listagem 38 – Método para salvar as formulações/alimento

```

@PostMapping("/{id}")
@ResponseBody
public ModelAndView salvar(@RequestBody @Valid List<FormulacaoAlimento>
    formulacaoAlimentoList, @PathVariable Long id, BindingResult result, Model
    model, RedirectAttributes attributes) {
    LocalDate hoje = LocalDate.now();
    LocalDate dataAnterior = hoje.plusDays(-2);
    List<Exigencia> exigenciaList =
    exigenciaRepository.findByAnimalFaseAnimalLactacaoPrePartoSeca(hoje,
    dataAnterior, id, FaseAnimal.LACTACAO.toString(),
    FaseAnimal.PREPARTO.toString(), FaseAnimal.SECA.toString());
}

```

```

        for (Exigencia exigencia : exigenciaList) {
            Formulacao formulacao = new Formulacao();
            formulacao.setDataFormulacao(LocalDate.now());
            formulacao.setAnimal(animalRepository.getOne(exigencia.getAnimal()
                .getId()));
            formulacaoRepository.save(formulacao);
            for (FormulacaoAlimento fa : formulacaoAlimentoList) {
                FormulacaoAlimento formulacaoAlimento = new
FormulacaoAlimento();
                formulacaoAlimento.setAlimento(fa.getAlimento());
                formulacaoAlimento.setPreco(fa.getPreco());
                formulacaoAlimento.setQuantidade(fa.getQuantidade());
                formulacaoAlimento.setFormulacao(formulacao);
                formulacaoAlimento.setDataEscolhaAlimento(LocalDate.now());
                formulacaoAlimentoRepository.save(formulacaoAlimento);
            }
            attributes.addFlashAttribute("mensagem", "Registro salvo com
sucesso!");
            return new
ModelAndView("redirect:/formulacaoAlimentoController/novo");
        }
    }

```

Fonte: Autoria própria.

No *controller* de formulação/alimento, o método “novo” da Listagem 39, recupera a lista das últimas exigências de cada animal e a lista de formulação/alimento em que a quantidade de alimento esteja nulo, ou seja, ainda não foram utilizadas para os cálculos de balanceamento.

Listagem 39 – Lista as formulações/alimentos

```

@GetMapping("/novo")
public ModelAndView novo(Formulacao formulacao, FormulacaoAlimento
    formulacaoAlimento, HttpSession session) {
    ModelAndView modelAndView = new
ModelAndView("balanceamento/balanceamento");
    LocalDate hoje = LocalDate.now();
    LocalDate dataAnterior = hoje.plusDays(-2);
    modelAndView.addObject(formulacaoAlimento);
    modelAndView.addObject(formulacao);
    Long idPropriedade = (Long)
session.getAttribute("idPropriedadeAtual");
    modelAndView.addObject("exigenciaList",
exigenciaRepository.findByAnimalFaseAnimalLactacaoPrePartoSeca
(hoje, dataAnterior, idPropriedade, FaseAnimal.LACTACAO.toString(),
FaseAnimal.PREPARTO.toString(), FaseAnimal.SECA.toString()));
    modelAndView.addObject("formulacaoAlimentoList",
formulacaoAlimentoRepository
    .findByFormulacaoAlimentoQuantidadeIsNullPropriedade
(idPropriedade));
    return modelAndView;
}

```

Fonte: Autoria própria.

O método apresentado na Listagem 40, realiza o cálculo dos nutrientes oferecidos, conforme o alimento escolhido e sua quantidade. Para cada nutriente diferente foi desenvolvido um método específico, que faz parte da estrutura apresentada na Listagem 40. Neles, é multiplicada a quantidade de alimento pela quantidade de nutrientes que possui, conforme cadastrado na tabela de “Alimentos” no banco de dados, gerando o valor fornecido em nutrientes.

Listagem 40 – Cálculo dos nutrientes oferecidos

```
@PostMapping("/calcular")
@ResponseBody
public NutrientesOferecidos calcular(@RequestBody @Valid
List<FormulacaoAlimento> formulacaoAlimentoList, BindingResult result,
Model model, RedirectAttributes attributes) {

    Double oferecidoIngestaoMateriaSeca = 0.0,
oferecidoNutrientesDigestiveisTotais = 0.0, oferecidoProteinaBruta = 0.0,
    oferecidoCalcio = 0.0, oferecidoFosforo = 0.0,
    oferecidoEnergiaLiquida = 0.0, oferecidoEnergiaMetabolizavel = 0.0,
    oferecidoProteinaDegradavelRumen = 0.0,
    oferecidoProteinaNaoDegradavelRumen = 0.0,
oferecidoProteinaMetabolizavel = 0.0,
    oferecidoFibraDetergenteNeutraEfetiva = 0.0,
oferecidoFibraDetergenteNeutro = 0.0, oferecidoExtratoEterio= 0.0;

    NutrientesOferecidos nutrientesOferecidos = new
NutrientesOferecidos();

    for (FormulacaoAlimento fa : formulacaoAlimentoList) {
        FormulacaoAlimento formulacaoAlimento =
formulacaoAlimentoRepository.getOne(fa.getId());
        formulacaoAlimento.setQuantidade(fa.getQuantidade());
        Alimento alimento = alimentoRepository
.getOne(formulacaoAlimento.getAlimento().getId());
        oferecidoIngestaoMateriaSeca = oferecidoIngestaoMateriaSeca +
oferecidoIngestaoMateriaSeca(formulacaoAlimento, alimento);
        oferecidoNutrientesDigestiveisTotais =
oferecidoNutrientesDigestiveisTotais + oferecidoNutrientesDigestiveisTotais
(formulacaoAlimento, alimento);
        oferecidoProteinaBruta = oferecidoProteinaBruta +
oferecidoProteinaBruta(formulacaoAlimento, alimento);
        oferecidoCalcio = oferecidoCalcio +
oferecidoCalcio(formulacaoAlimento, alimento);
        oferecidoFosforo = oferecidoFosforo +
oferecidoFosforo(formulacaoAlimento, alimento);
        oferecidoEnergiaLiquida = oferecidoEnergiaLiquida +
oferecidoEnergiaLiquida(formulacaoAlimento, alimento);
        oferecidoEnergiaMetabolizavel = oferecidoEnergiaMetabolizavel +
oferecidoEnergiaMetabolizavel(formulacaoAlimento, alimento);
        oferecidoProteinaDegradavelRumen =
oferecidoProteinaDegradavelRumen +
oferecidoProteinaDegradavelRumen(formulacaoAlimento, alimento);
        oferecidoProteinaNaoDegradavelRumen =
oferecidoProteinaNaoDegradavelRumen
+ oferecidoProteinaNaoDegradavelRumen(formulacaoAlimento, alimento);
        oferecidoProteinaMetabolizavel = oferecidoProteinaMetabolizavel
```

```

+ oferecidoProteinaMetabolizavel(formulacaoAlimento, alimento);
    oferecidoFibraDetergenteNeutraEfetiva =
oferecidoFibraDetergenteNeutraEfetiva
+ oferecidoFibraDetergenteNeutraEfetiva(formulacaoAlimento, alimento);
    oferecidoFibraDetergenteNeutro = oferecidoFibraDetergenteNeutro
+ oferecidoFibraDetergenteNeutro(formulacaoAlimento, alimento);
    oferecidoExtratoEterio = oferecidoExtratoEterio
+ oferecidoExtratoEterio(formulacaoAlimento, alimento);
    }
    nutrientesOferecidos
.setIngestaoMateriaSeca(oferecidoIngestaoMateriaSeca);
    nutrientesOferecidos
.setNutrientesDigestiveisTotaisKg(oferecidoNutrientesDigestiveisTotais);
    nutrientesOferecidos.setProteinaBrutaKg(oferecidoProteinaBruta);
    nutrientesOferecidos.setCalcioKg(oferecidoCalcio);
    nutrientesOferecidos.setFosforoKg(oferecidoFosforo);
    nutrientesOferecidos
.setEnergiaLiquidaMcalKg(oferecidoEnergiaLiquida);
    nutrientesOferecidos
.setEnergiaMetabolizavelMcalKg(oferecidoEnergiaMetabolizavel);
    nutrientesOferecidos
.setProteinaDegradavelRumenKg(oferecidoProteinaDegradavelRumen);
    nutrientesOferecidos
.setProteinaNaoDegradavelRumenKg(oferecidoProteinaNaoDegradavelRumen);
    nutrientesOferecidos
.setProteinaMetabolizavelKg(oferecidoProteinaMetabolizavel);
    nutrientesOferecidos
.setFibraDetergenteNeutroEfetivaPorcentagem
(oferecidoFibraDetergenteNeutraEfetiva);
    nutrientesOferecidos
.setFibraDetergenteNeutroPorcentagem(oferecidoFibraDetergenteNeutro);
    nutrientesOferecidos
.setExtratoEtereoporcentagem(oferecidoExtratoEterio);

    attributes.addFlashAttribute("mensagem", "Registro salvo com
sucesso!");
    return nutrientesOferecidos;
}

```

Fonte: Autoria própria.

Para encontrar a quantidade ideal de alimento, o usuário tem que testar o valor algumas vezes. Sendo assim, para deixar o processo mais dinâmico possível, foi desenvolvido o código em JavaScript apresentado na Listagem 41. Quando o usuário digita e pressiona a tecla Enter, é criado um *array* de objetos de formulação/alimento e enviado, via requisição AJAX, para o servidor, que realiza o cálculo e retornando um objeto de nutrientes oferecidos. A linha da tabela é criada dinamicamente com os valores do objeto.

Listagem 41 – Requisição Ajax de formulação/alimento

```

$('document').ready(function(e) {

```

```

    $('#form[id] div div input').keydown(function(e){
        if(e.which == 13 ){
            e.preventDefault();
            var id = $("#carouselBalanceamento .active").attr("id");
            let objAlimentos = carregarQuantidadeAlimento(id);
            $.ajax({
                headers : {
                    Accept : 'application/json'
                },
                type : $('#formQuantidadeAlimento_' + id).attr('method'),
                url : $('#formQuantidadeAlimento_' + id).attr('action'),
                data : JSON.stringify(objAlimentos),
                contentType:'application/json',
                success : function(data) {
                    if($('##tabelaNutrientesOferecidos_' + id + ' tbody tr').length > 0){
                        $('##tabelaNutrientesOferecidos_' + id + ' tbody tr').remove();
                    }

                    let itens = "";
                    itens += "<tr>";
                    itens += "<td>" + data.fibraDetergenteNeutroPorcentagem.toFixed(2) + "
                    </td>";
                    itens += "<td>" + data.extratoEtereoPorcentagem.toFixed(2) + " </td>";
                    itens += "<td>" + data.fibraDetergenteNeutroEfetivaPorcentagem.toFixed(2) + "
                    </td>";
                    itens += "<td>" + data.energiaMetabolizavelMcalKg.toFixed(2) + " </td>";
                    itens += "<td>" + data.ingestaoMateriaSeca.toFixed(2) + "</td>";
                    itens += "<td>" + data.energiaLiquidaMcalKg.toFixed(2) + " </td>";
                    itens += "<td>" + data.nutrientesDigestiveisTotaisKg.toFixed(2) + "</td>";
                    itens += "<td>" + data.proteinaMetabolizavelKg.toFixed(2) + "</td>";
                    itens += "<td>" + data.proteinaBrutaKg.toFixed(2) + "</td>";
                    itens += "<td>" + data.proteinaDegradavelRumenKg.toFixed(2) + "</td>";
                    itens += "<td>" + data.proteinaNaoDegradavelRumenKg.toFixed(2) + "</td>";
                    itens += "<td>" + data.calcioKg.toFixed(2) + "</td>";
                    itens += "<td>" + data.fosforoKg.toFixed(2) + "</td>";
                    itens += "</tr>";

                    $('##tabelaNutrientesOferecidos_' + id).append( itens );

                },
                error : function() {
                    alert('Error !!!!');
                }
            });
        }
    })
}

```

Fonte: Autoria própria.

Quando o usuário avalia que as quantidades de alimentos oferecidos estão adequadas, ele conclui o balanceamento daquele animal, chamando o método “salvar” apresentado na Listagem 42, que vai buscar a formulação/alimento que está armazenada no banco de dados e atualizar o valor da quantidade, salvando novamente.

Listagem 42 – Método para salvar as quantidades de alimentos

```

@PostMapping("/{idAnimal}")
public ResponseEntity<?> salvar(@RequestBody @Valid
List<FormulacaoAlimento> formulacaoAlimentoList, @PathVariable Long
idAnimal, BindingResult result, Model model, RedirectAttributes attributes)
{
    if (result.hasErrors()) {
        return new ResponseEntity<>(result.getAllErrors(),
HttpStatus.BAD_REQUEST);
    }
    NutrientesOferecidos nutrientesOferecidos = new
NutrientesOferecidos();
    nutrientesOferecidos = calcular(formulacaoAlimentoList,
result, model, attributes);
    for (FormulacaoAlimento fa : formulacaoAlimentoList) {
        FormulacaoAlimento formulacaoAlimento =
formulacaoAlimentoRepository.findByFormulacaoAlimentoIdAndAlimento
(fa.getId());
        formulacaoAlimento.setQuantidade(fa.getQuantidade());
        formulacaoAlimentoRepository.save(formulacaoAlimento);
        nutrientesOferecidos.setFormulacao(formulacaoAlimento
.getFormulacao());
    }
    nutrientesOferecidosRepository.save(nutrientesOferecidos);
    attributes.addFlashAttribute("mensagem", "Registro salvo com
sucesso!");
    return new ResponseEntity<>(HttpStatus.OK);
}

```

Fonte: Autoria própria.

Para gerar os gráficos de peso e produção de leite foi utilizada a biblioteca do Google Chart. No servidor, o método “listaPesos12meses” busca no banco de dados os pesos lançados nos últimos 12 meses, retornando a lista dos registros encontrados, conforme apresentado na Listagem 43.

Listagem 43 – Cria lista de pesos do animal

```

@GetMapping("/{listaPesos/{idAnimal}}")
@ResponseBody
private List<PesoAnimal> listaPesos12meses(@PathVariable Long
idAnimal){
    LocalDate hoje = LocalDate.now();
    List<PesoAnimal> peso =
pesoAnimalRepository.findByPesoAnimalUltimos12Meses(idAnimal,hoje,
hoje.plusDays(-365) );
    return peso;
}

```

Fonte: Autoria própria.

No lado cliente o JavaScript apresentado na Listagem 44 recebe os dados, cria a tabela, as colunas e as linhas, gerando o gráfico e mostrando no elemento HTML definido. Para o gráfico de lactação, foi utilizado o mesmo padrão.

Listagem 44 – Gerador do gráfico

```
$(window).on("load",function(e){

    e.preventDefault();
    let id = $('#chartPeso').attr('id');
    console.log(id);
    google.charts.load('current', {'packages':['corechart']});
    google.charts.setOnLoadCallback(drawChart);

    function drawChart() {
        $.ajax({
            headers : {
                Accept : 'application/json'
            },
            url: "/animaisController/listaPesos/"+id,
            dataType: "json"

        }).done(function(jsonData){

            var data = new
google.visualization.DataTable(jsonData);
            data.addColumn('string', 'Data');
            data.addColumn('number', 'Peso');

            jsonData.forEach(function (row) {
                data.addRow([
                    row.dataPesagem,
                    row.peso

                ]);
            });

            var chart = new
google.visualization.LineChart(document.getElementById('chartPeso'));
            chart.draw(data, {width:600, height: 500});
        })

    }

})
```

Fonte: Autoria própria.

CONCLUSÃO

O objetivo deste trabalho foi o desenvolvimento de um sistema *web* que armazene os dados coletados para realizar o balanceamento nutricional dos animais. O sistema é alimentado com dados dos animais desde seu nascimento, passando por todas as fases da vida, estipulando a alimentação adequada para cada fase, baseada no tipo de alimento disponível. Tornando mais fácil o acesso, armazenamento e consulta de dados, e posterior análise e tomada de decisão.

A utilização de *Thymeleaf* e *Spring MVC* se apresentou muito efetiva quanto a integração servidor e cliente. O *Thymeleaf* possibilita o reaproveitamento de código HTML, por meio dos *fragments*, tornando o desenvolvimento mais ágil. A utilização do *Spring* no desenvolvimento, torna as requisições HTTP muito mais simples no seu entendimento e estruturação, permitindo focar na criação das regras de negócio.

Os desafios encontrados durante o desenvolvimento, estão principalmente na realização das diversas consultas ao banco, em que é necessário trazer o último registro, ou seja, com a data mais recente, de uma tabela.

Conforme os requisitos funcionais e não-funcionais levantados junto à Instituição de Pesquisa, também analisando as planilhas utilizadas no balanceamento, pode-se concluir que o sistema atende ao objetivo de gerenciar o rebanho no que se refere as inseminações, reprodução, pesos e produção de leite. Também é possível fazer o balanceamento entre a exigência e os nutrientes oferecidos, gerando o relatório final com as quantidades de alimento.

Como sugestão para futuros trabalhos, pode-se buscar desenvolver um algoritmo que encontre a quantidade adequada de alimento de forma automática, sem necessitar que o usuário digite os valores. Essa possibilidade existe com a programação linear, que resolve problemas de otimização. Ou seja, quanto de cada alimento disponível é necessário fornecer ao animal para atender à exigência nutricional dele, podendo inclusive, considerar o preço do alimento, para definir as quantidades.

REFERÊNCIAS

- BIANCHINI, Sandro Lopes. **Avaliação de métodos de desenvolvimento de aplicações web.** Dissertação de mestrado. USP, 2008. Disponível em: <<http://www.teses.usp.br/teses/disponiveis/55/55134/tde-01072008-143726/pt-br.php>>. Acesso em: 18 jun. 2018
- COSTA, Ana Luiza da; SILVA, Ricardo Reis e; LAGE, Helena F.; FERREIRA, Alexandre. **Exigências nutricionais de bovinos leiteiros no Brasil.** Disponível em: <<http://www.revistaleiteintegral.com.br/noticia/exigencias-nutricionais-de-bovinos-leiteiros-no-brasil>> Acesso em: 15 jun. 2018.
- DURELLI, Vinicius H. S.; VIANA, Matheus Carvalho; PENTEADO, Rosângela. **Uma proposta de reúso de interface gráfica com o usuário baseada no padrão arquitetural MVC.** Disponível em: <<file:///C:/Users/Vanessa%20Lucion/Downloads/SBSI-2008-DURELLI-ARTIGO.pdf>> Acesso em: 16 jun. 2018.
- LAZIA, Beatriz. **Saiba como fazer o balanceamento da dieta de vacas leiteiras.** Disponível em: <<http://www.portalogropecuário.com.br/bovinos/pecuaria-de-leite/saiba-como-fazer-balanceamento-dieta-vacas-leiteiras/>> Acesso em: 16 jun. 2018.
- LUNA, Andréia Miranda de. **Geração de Interfaces RIA dirigida por ontologias.** Disponível em: <https://www.maxwell.vrac.puc-rio.br/15686/15686_3.PDF> Acesso em: 18 jun. 2018.
- MATUZAKI, Adauto. **Aplicação web usando interfaces ricas para a gestão de clínicas de enfermagem.** Disponível em: <<https://cepein.femanet.com.br/BDigital/arqTccs/1011330523.pdf>> Acesso em: 18 jun. 2018.
- MDA - SECRETARIA ESPECIAL DE AGRICULTURA FAMILIAR E DO DESENVOLVIMENTO AGRÁRIO. **O que é a agricultura familiar.** Disponível em <<http://www.mda.gov.br/sitemda/noticias/o-que-%C3%A9-agricultura-familiar>>. Acesso em: 26 mar. 2018.
- OLIVEIRA JUNIOR, Edson Alves de; FORTES, Renata Pontin de Mattos. **Arquitetura de software na web atual: Processamento no Servidor.** Disponível em: <http://conteudo.icmc.usp.br/CMS/Arquivos/arquivos_enviados/BIBLIOTECA_113_ND_78.pdf> Acesso em: 16 jun. 2018.
- PODESTÀ, Inez de. **Agropecuária puxa o PIB de 2017.** Disponível em <<http://www.agricultura.gov.br/noticias/agropecuaria-puxa-o-pib-de-2017>>. Acesso em: 26 mar. 2018.
- PRESSMAN, Roger. **Engenharia de software.** Rio de Janeiro: McGraw-Hill, 2008.
- PUREWAL, Semmy **Aprendendo a desenvolver aplicações web:** Desenvolva rapidamente com as tecnologias JavaScript mais modernas. São Paulo: Novatec, 2014.

SALMAN, Ana Karina Dias; OSMARI, Elisa Köhler; SANTOS, Márcio Gregório Rojas dos. **Manual prático para formulação de ração para vacas leiteiras**. Disponível em: <<https://ainfo.cnptia.embrapa.br/digital/bitstream/item/66779/1/doc145-vacasleiteiras-1.pdf>> Acesso em: 15 jun. 2018.

SMEETS, Bram; BANKRAS, Roald. **Programando Google Web Toolkit: Do Iniciante ao Profissional**. Disponível em: <<http://www.altabooks.com.br/programando-google-web-toolkit-do-iniciante-ao-profissional.html>> Acesso em: 18 jun. 2018.

TOMICH, Thierry R.; MACHADO, Fernanda S.; PEREIRA, Luiz G. R.; CAMPOS, Mariana Magalhães. **Nutrição de precisão na pecuária leiteira**. Disponível em <<https://ainfo.cnptia.embrapa.br/digital/bitstream/item/139557/1/Cnppl-2015-CadTecVetZoot-Nutricao.pdf>> Acesso em: 15 jun. 2018.

ZOCCAL, Rosangela. **A força do agro e do leite no Brasil**. Disponível em <<http://www.repilite.com.br/profiles/blogs/a-for-a-do-agro-e-do-leite-no-brasil>>. Acesso em: 27 mar. 2018.